CASO DE ESTUDIO 2 / ENTREGABLE 03

Réplica en miniatura

Sistema de recompensas dinámico de MiMcDonald's

Compilación de los cinco notebooks

La réplica reúne el código, las explicaciones y los resultados guardados de las cinco etapas. Los datos son sintéticos. Esta edición permite revisar la ejecución; los notebooks del repositorio conservan el formato ejecutable.

Integrante	Carné
Nelson García Bravatti	22434
Ricardo Chuy	221007
Diego Valenzuela	22309
Daniel Dubón	22233
Sergio Orellana	221122
Joaquín Puente	22296

Repositorio de GitHub

Machine Learning Engineering · Universidad del Valle de Guatemala

Contenido

01 Generación de datos sintéticos

02 Limpieza de Bronze a Silver

03 Agregación de Silver a Gold

04 Segmentación con K-means

05 Reglas, movimientos de puntos e insights

Procedimiento de la prueba funcional

Cada paso corresponde a un notebook; ejecutarlos en orden equivale a correr el pipeline completo.

1. **Generación (01):** crear los CSV sintéticos en `data/bronze/`.
2. **Silver (02):** limpiar y validar los datos con DuckDB; guardar las tablas limpias en `data/silver/` y los registros rechazados en `data/cuarentena/`.
3. **Gold (03):** con los meses 1 y 2, calcular `perfil_clientes` (sin segmento) y `ventas_productos`.
4. **ML (04):** segmentar a los clientes, completar la columna `segmento` y calcular `afinidad_segmento_categoria` con los meses 1 y 2 (sección 4).
5. **Puntos e insights (05):** generar `reglas_puntos`, calcular `movimientos_puntos` (regla fija en los meses 1 y 2, reglas dinámicas en el mes 3) y comparar ambos sistemas en el mes 3.

Fórmula por línea de compra:

puntos_base = ceil(subtotal × 10)

multiplicador_aplicado = min(multiplicador_producto × multiplicador_cliente, 3)

puntos = ceil(subtotal × 10 × multiplicador_aplicado)

Se multiplica el importe sin redondear y se redondea al final de cada línea. No se multiplica `puntos_base` ya redondeado. Se mantiene la tasa de 10 puntos por quetzal y ningún multiplicador reduce la acumulación. El cálculo por línea es una convención de la simulación.

Propuesta operativa: el plazo público de acreditación permite considerar procesamiento por lotes, siempre que su frecuencia, duración y manejo de fallos respeten las 24 horas. La POC acredita al terminar cada compra para reconstruir saldos; no implementa un servicio diario. En producción, el canje requeriría además una validación transaccional del saldo disponible.

Preparación del entorno

Desde la carpeta del caso, se instalan las dependencias y se inicia Jupyter:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m jupyter lab
```

La ejecución sigue el orden 01 a 05 con el mismo entorno. Si cambia una etapa, se ejecutan nuevamente las posteriores.

01 - Generación de datos sintéticos (Bronze)

Generamos cinco archivos CSV para simular las fuentes del sistema de puntos. La base válida contiene 1,000 clientes, 28 productos, 10 restaurantes y 20,000 compras distribuidas en tres meses. Los nombres, correos, restaurantes, precios y comportamientos son sintéticos.

Utilizamos la biblioteca estándar de Python y una semilla fija para reproducir los datos. Primero validamos las transacciones y después incorporamos errores controlados que permiten evaluar la limpieza. La ejecución sigue el orden de las celdas.

Código · celda 02

```
import calendar
import csv
import hashlib
import random
from collections import Counter, defaultdict
from datetime import date, datetime, time, timedelta
from decimal import Decimal
from pathlib import Path

# Localizar la carpeta raíz del caso.
PROJECT_ROOT = next(
    (path for path in (Path.cwd(), *Path.cwd().parents) if (path / "doc.md").is_file()),
    None,
)
if PROJECT_ROOT is None:
    raise RuntimeError("No se encuentra la carpeta del caso. Inicie la ejecución desde Caso-Estudio-2.")
BRONZE = PROJECT_ROOT / "data" / "bronze"
SEED = 22434
YEAR = 2025
POINTS_PER_QUETZAL = 10
ANALYSIS_END = date(YEAR, 2, 28)
PERIOD_START = date(YEAR, 1, 1)
PERIOD_END = date(YEAR, 3, 31)
CATEGORIES = (
    "hamburguesas", "pollo", "desayunos", "acompañamientos",
    "bebidas", "postres", "ensaladas",
)
PROFILES = ("Leales", "Madrugadores", "En riesgo", "Ocasionales")
SCHEMAS = {
    "clientes": ["id", "nombre", "correo", "fecha_registro"],
    "productos": ["id", "nombre", "categoria", "precio_normal", "precio_en_puntos"],
    "restaurantes": ["id", "nombre", "zona"],
    "compras": ["id", "cliente_id", "restaurante_id", "fecha_hora", "monto_total", "forma_pago"],
    "lineas_compra": ["id", "compra_id", "producto_id", "cantidad", "precio_unitario", "subtotal", "pagado_con_puntos"],
}
```

```
def money(cents):
    return f"{Decimal(cents) / 100:.2f}"


def cents(value):
    amount = Decimal(str(value)) * 100
    assert amount == amount.to_integral_value(), "Monto con más de dos decimales"
    return int(amount)


def base_points(amount_cents):
    return (amount_cents * POINTS_PER_QUETZAL + 99) // 100


assert base_points(1235) == 124
assert base_points(5000) == 500
print("Generación de datos sintéticos.")
print(f"Período: {PERIOD_START} a {PERIOD_END}; semilla: {SEED}")
```

```
Generación de datos sintéticos.
Período: 2025-01-01 a 2025-03-31; semilla: 22434
```

1. Clientes, catálogo y restaurantes

Distribuimos al azar a los clientes entre cuatro perfiles de comportamiento. Las etiquetas ocultas orientan la generación y la revisión del conjunto sintético; los CSV contienen únicamente las variables observables del cliente y sus compras.

Cada categoría reúne cuatro productos. Asignamos una probabilidad de selección menor a cuatro productos de categorías distintas para simular baja rotación. Todos los clientes tienen una fecha de registro anterior al inicio del período.

Código · celda 04

```
# Reiniciar esta celda reinicia toda la generación determinista.
rng = random.Random(SEED)
customer_ids = list(range(1, 1001))
rng.shuffle(customer_ids)
hidden_profiles = {
    customer_id: PROFILES[index // 250]
    for index, customer_id in enumerate(customer_ids)
}
clients = [
    {
        "id": customer_id,
        "nombre": f"Cliente Sintético {customer_id:04d}",
        "correo": f"cliente{customer_id:04d}@example.com",
        "fecha_registro": (PERIOD_START - timedelta(days=rng.randint(1, 365))).isoformat(),
    }
```

```
    for customer_id in sorted(customer_ids)
]
# Precios en centavos; se incluyen fracciones para ejercitar el redondeo.
menu = {
    "hamburguesas": [("Big Mac", 3595), ("Cuarto de libra", 3890), ("Hamburguesa con queso", 1800), ("Hamburguesa especial", 3250)],
    "pollo": [("McPollo", 3200), ("McNuggets 6", 2850), ("Pollo crujiente", 3500), ("Sándwich de pollo especial", 3695)],
    "desayunos": [("McMuffin huevo", 2450), ("McMuffin salchicha", 2695), ("Hotcakes", 2200), ("Desayuno especial", 3100)],
    "acompañamientos": [("Papas pequeñas", 1235), ("Papas medianas", 1650), ("Papas grandes", 1995), ("Papas especiales", 2250)],
    "bebidas": [("Café", 1095), ("Gaseosa", 1200), ("Jugo de naranja", 1550), ("Agua", 1000)],
    "postres": [("McFlurry Oreo", 2495), ("Sundae chocolate", 1800), ("Cono", 800), ("Pie de manzana", 1500)],
    "ensaladas": [("Ensalada de pollo", 3500), ("Ensalada verde", 2600), ("Ensalada mixta", 2950), ("Ensalada especial", 3295)],
}
products = []
product_cents = {}
for category, items in menu.items():
    for name, price in items:
        product_id = len(products) + 1
        products.append({
            "id": product_id, "nombre": name, "categoria": category,
            "precio_normal": money(price), "precio_en_puntos": (price * 60 + 99) // 100,
        })
        product_cents[product_id] = price
laggard_ids = {4, 8, 24, 28}
product_by_id = {product["id"]: product for product in products}
products_by_category = {
    category: [product["id"] for product in products if product["categoria"] == category]
    for category in CATEGORIES
}
restaurants = [
    {"id": index, "nombre": f"Restaurante Sintético {index:02d}", "zona": zone}
    for index, zone in enumerate(
        ["Zona 1", "Zona 4", "Zona 7", "Zona 9", "Zona 10", "Zona 11", "Zona 12", "Zona 15", "Mixco", "Villa Nueva"], 1
    )
]
print(f"Clientes: {len(clients)}; productos: {len(products)}; restaurantes: {len(restaurants)}")
print("Perfiles ocultos:", dict(Counter(hidden_profiles.values())))
```

```
Clientes: 1000; productos: 28; restaurantes: 10
Perfiles ocultos: {'Leales': 250, 'Madrugadores': 250, 'En riesgo': 250, 'Ocasionales': 250}
```

2. Calendario de compras y preferencias

Distribuimos 20,000 compras con variación individual: 9,000 de Leales, 6,500 de Madrugadores, 3,500 de En riesgo y 1,000 de Ocasionales. Los clientes En riesgo concentran sus compras en enero y reducen su actividad en febrero, cuando solo compran durante la primera semana.

Los Ocasionales realizan una o dos compras al mes. Los Madrugadores compran principalmente entre las 06:00 y las 11:00. Cada perfil presenta preferencias por categoría. Ordenamos el calendario cronológicamente antes de simular los saldos y canjes.

Código · celda 06

```
monthly_targets = {
    "Leales": [3000, 3000, 3000],
    "Madrugadores": [2166, 2167, 2167],
    "En riesgo": [3000, 250, 250],
    "Ocasionales": [333, 333, 334],
}
# Pesos en el orden definido en CATEGORIES.
category_weights = {
    "Leales": [30, 25, 5, 12, 8, 17, 3],
    "Madrugadores": [3, 2, 51, 3, 35, 3, 3],
    "En riesgo": [20, 15, 8, 8, 8, 8, 33],
    "Ocasionales": [8, 5, 5, 30, 35, 15, 2],
}
# Franjas: mañana [06,11), almuerzo [11,15), tarde [15,18), noche [18,23).
time_windows = [(6, 11), (11, 15), (15, 18), (18, 23)]
hour_weights = {
    "Leales": [25, 25, 25, 25],
    "Madrugadores": [97, 1, 1, 1],
    "En riesgo": [10, 40, 15, 35],
    "Ocasionales": [10, 20, 40, 30],
}
calendar_events = []
for profile in PROFILES:
    members = sorted(customer_id for customer_id in customer_ids if hidden_profiles[customer_id] == profile)
    activity_weights = [rng.uniform(0.4, 1.6) for _ in members]
    for month, target in enumerate(monthly_targets[profile], 1):
        if profile == "Ocasionales":
            visits = members + rng.sample(members, target - len(members))
        else:
            minimum = 4 if profile == "Leales" else 3 if profile == "Madrugadores" or month == 1 else 0
            visits = members * minimum
            visits += rng.choices(members, weights=activity_weights, k=target - len(visits))
        for customer_id in visits:
            max_day = 7 if profile == "En riesgo" and month == 2 else calendar.monthrange(YEAR, month)[1]
            day = rng.randint(1, max_day)
            start_hour, end_hour = rng.choices(time_windows, weights=hour_weights[profile])[0]
            seconds = rng.randrange((end_hour - start_hour) * 3600)
            timestamp = datetime(YEAR, month, day, start_hour) + timedelta(seconds=seconds)
            calendar_events.append((timestamp, customer_id, rng.randint(1, len(restaurants))))
calendar_events.sort()
assert len(calendar_events) == 20_000
print("Calendario listo:", len(calendar_events), "compras")
```

```
Calendario listo: 20000 compras
```

3. Líneas de compra y canjes

Cada compra contiene de una a cuatro líneas con productos distintos. Los Leales presentan más líneas y unidades para representar tickets mayores. Asignamos a los productos rezagados un peso de selección de `0.025`, frente a `1` para los demás productos disponibles de su categoría.

Representamos el dinero en centavos enteros para evitar errores de punto flotante. Permitimos canjes únicamente cuando el cliente cuenta con saldo suficiente antes de la compra. Primero descontamos los canjes y después acreditamos los puntos de las líneas monetarias. Este saldo permite generar transacciones coherentes y sirve como referencia para reconstruir el historial de puntos.

Código · celda 08

```
purchases = []
lines = []
balances = defaultdict(int)
for purchase_id, (timestamp, customer_id, restaurant_id) in enumerate(calendar_events, 1):
    profile = hidden_profiles[customer_id]
    count_weights = [5, 20, 40, 35] if profile == "Leales" else [35, 40, 20, 5] if profile == "Madrugadores" else [65, 25, 8, 2] if profile == "Ocasionales" else [20, 40, 30,
    10]
    number_of_lines = rng.choices([1, 2, 3, 4], weights=count_weights)[0]
    selected = set()
    purchase_total = 0
    points_earned = 0
    for _ in range(number_of_lines):
        category = rng.choices(CATEGORIES, weights=category_weights[profile])[0]
        candidates = [product_id for product_id in products_by_category[category] if product_id not in selected]
        product_id = rng.choices(candidates, weights=[0.025 if pid in laggard_ids else 1 for pid in candidates])[0]
        selected.add(product_id)
        quantity = rng.choices([1, 2, 3], weights=[65, 30, 5])[0] if profile == "Leales" else 1
        redemption_cost = product_by_id[product_id]["precio_en_puntos"] * quantity
        redeemed = balances[customer_id] >= redemption_cost and rng.random() < 0.06
        unit_price = 0 if redeemed else product_cents[product_id]
        subtotal = unit_price * quantity
        if redeemed:
            balances[customer_id] -= redemption_cost
        else:
            points_earned += base_points(subtotal)
        purchase_total += subtotal
        lines.append({
            "id": len(lines) + 1, "compra_id": purchase_id, "producto_id": product_id,
            "cantidad": quantity, "precio_unitario": money(unit_price), "subtotal": money(subtotal),
            "pagado_con_puntos": redeemed,
        })
    balances[customer_id] += points_earned
    purchases.append({
```

```
        "id": purchase_id, "cliente_id": customer_id, "restaurante_id": restaurant_id,
        "fecha_hora": timestamp.isoformat(sep=" "), "monto_total": money(purchase_total),
        "forma_pago": rng.choice(["efectivo", "tarjeta"]),
    })
clean_tables = {
    "clientes": clients, "productos": products, "restaurantes": restaurants,
    "compras": purchases, "lineas_compra": lines,
}
print("Base válida:", {name: len(rows) for name, rows in clean_tables.items()})
print("Líneas canjeadas:", sum(line["pagado_con_puntos"] for line in lines))
```

```
Base válida: {'clientes': 1000, 'productos': 28, 'restaurantes': 10, 'compras': 20000, 'lineas_compra': 49525}
Líneas canjeadas: 2495
```

4. Validación de la base válida y de los patrones

Comprobamos esquemas, claves únicas, relaciones, fechas, montos, granularidad, redondeo y saldos cronológicos. También revisamos los patrones de la ventana de análisis y verificamos que cuatro productos presenten un índice de ventas menor a 0.5. Estas comprobaciones confirman la coherencia del conjunto sintético antes de introducir errores.

Código · celda 10

```
def validate_clean_tables(tables):
    for name, rows in tables.items():
        assert all(list(row) == SCHEMAS[name] for row in rows), name
        assert len({row["id"] for row in rows}) == len(rows), f"ID repetido: {name}"
    customers = {row["id"]: row for row in tables["clientes"]}
    catalog = {row["id"]: row for row in tables["productos"]}
    stores = {row["id"] for row in tables["restaurantes"]}
    headers = {row["id"]: row for row in tables["compras"]}
    assert len({row["correo"] for row in customers.values()}) == len(customers)
    for row in customers.values():
        assert row["nombre"] and row["correo"]
        assert date.fromisoformat(row["fecha_registro"]) < PERIOD_START
    for row in catalog.values():
        assert row["categoria"] in CATEGORIES
        assert cents(row["precio_normal"]) > 0 and row["precio_en_puntos"] > 0
    grouped_lines = defaultdict(list)
    for line in tables["lineas_compra"]:
        assert line["compra_id"] in headers and line["producto_id"] in catalog
        assert isinstance(line["cantidad"], int) and line["cantidad"] >= 1
        assert isinstance(line["pagado_con_puntos"], bool)
        assert cents(line["subtotal"]) == line["cantidad"] * cents(line["precio_unitario"])
        assert cents(line["precio_unitario"]) >= 0
        if line["pagado_con_puntos"]:
            assert cents(line["precio_unitario"]) == 0
        else:
```

```
            assert cents(line["precio_unitario"]) == cents(catalog[line["producto_id"]]["precio_normal"])
        grouped_lines[line["compra_id"]].append(line)
    audited_balances = defaultdict(int)
    redeemed_lines = 0
    for purchase in sorted(headers.values(), key=lambda row: (row["fecha_hora"], row["id"])):
        customer_id = purchase["cliente_id"]
        assert customer_id in customers and purchase["restaurante_id"] in stores
        timestamp = datetime.fromisoformat(purchase["fecha_hora"])
        assert PERIOD_START <= timestamp.date() <= PERIOD_END
        assert time(6) <= timestamp.time() < time(23)
        assert date.fromisoformat(customers[customer_id]["fecha_registro"]) < timestamp.date()
        assert purchase["forma_pago"] in {"efectivo", "tarjeta"}
        details = grouped_lines[purchase["id"]]
        assert 1 <= len(details) <= 4
        assert len({line["producto_id"] for line in details}) == len(details)
        assert cents(purchase["monto_total"]) == sum(cents(line["subtotal"]) for line in details)
        assert cents(purchase["monto_total"]) >= 0
        earned = 0
        for line in details:
            if line["pagado_con_puntos"]:
                cost = catalog[line["producto_id"]]["precio_en_puntos"] * line["cantidad"]
                assert audited_balances[customer_id] >= cost, "Canje sin saldo previo"
                audited_balances[customer_id] -= cost
                redeemed_lines += 1
            else:
                earned += base_points(cents(line["subtotal"]))
        audited_balances[customer_id] += earned
    assert redeemed_lines > 0
    assert len(audited_balances) == 1000 and min(audited_balances.values()) >= 0
    return dict(audited_balances)


assert validate_clean_tables(clean_tables) == dict(balances)
# Los diagnósticos utilizan exclusivamente las compras de enero-febrero.
analysis_purchases = {row["id"]: row for row in purchases if date.fromisoformat(row["fecha_hora"][:10]) <= ANALYSIS_END}
analysis_units = Counter()
profile_category_visits = defaultdict(set)
for line in lines:
    purchase = analysis_purchases.get(line["compra_id"])
    if purchase is None:
        continue
    if not line["pagado_con_puntos"]:
        analysis_units[line["producto_id"]] += line["cantidad"]
    profile = hidden_profiles[purchase["cliente_id"]]
    profile_category_visits[(profile, product_by_id[line["producto_id"]]["categoria"])].add(purchase["id"])
laggard_indices = {}
for category, ids in products_by_category.items():
    category_average = sum(analysis_units[pid] for pid in ids) / len(ids)
```

```
    assert category_average > 0
    for product_id in ids:
        index = analysis_units[product_id] / category_average
        if index < 0.5:
            laggard_indices[product_id] = index
assert set(laggard_indices) == laggard_ids, laggard_indices
print("Rezagados en meses 1-2:")
for product_id, index in sorted(laggard_indices.items()):
    print(f"  {product_by_id[product_id]['nombre']}: índice {index:.3f}")

profile_stats = {}
for profile in PROFILES:
    rows = [row for row in purchases if hidden_profiles[row["cliente_id"]] == profile]
    monthly = [sum(row["fecha_hora"].startswith(f"{YEAR}-{month:02d}") for row in rows) for month in (1, 2, 3)]
    morning_share = sum(6 <= datetime.fromisoformat(row["fecha_hora"]).hour < 11 for row in rows) / len(rows)
    ticket = sum(cents(row["monto_total"]) for row in rows) / len(rows) / 100
    profile_stats[profile] = (monthly, morning_share, ticket)
    assert monthly == monthly_targets[profile]
    print(f"{profile:14s} | compras por mes {monthly} | mañana {morning_share:.1%} | ticket Q{ticket:.2f}")
assert profile_stats["Madrugadores"][1] > 0.90
assert profile_stats["En riesgo"][0][1] < profile_stats["En riesgo"][0][0] * 0.15
assert profile_stats["Leales"][2] > profile_stats["Ocasionales"][2]
occasional_counts = Counter((row["cliente_id"], row["fecha_hora"][:7]) for row in purchases if hidden_profiles[row["cliente_id"]] == "Ocasionales")
assert len(occasional_counts) == 750 and set(occasional_counts.values()) <= {1, 2}
# Afinidad observable en datos, sin exportar etiquetas de generación.
profile_analysis_counts = Counter(hidden_profiles[row["cliente_id"]] for row in analysis_purchases.values())
for category in CATEGORIES:
    all_visits = sum(len(profile_category_visits[(profile, category)]) for profile in PROFILES)
    overall_share = all_visits / len(analysis_purchases)
    assert overall_share > 0
    affinity = {
        profile: (len(profile_category_visits[(profile, category)]) / profile_analysis_counts[profile]) / overall_share
        for profile in PROFILES
    }
    assert max(affinity.values()) > 1.2, (category, affinity)
print("Validación correcta: integridad, importes, canjes, saldos y patrones de comportamiento.")
```

```
Rezagados en meses 1-2:
  Hamburguesa especial: índice 0.064
  Sándwich de pollo especial: índice 0.052
  Pie de manzana: índice 0.034
  Ensalada especial: índice 0.046
Leales         | compras por mes [3000, 3000, 3000] | mañana 24.4% | ticket Q102.07
Madrugadores   | compras por mes [2166, 2167, 2167] | mañana 97.3% | ticket Q38.67
En riesgo      | compras por mes [3000, 250, 250] | mañana 10.8% | ticket Q59.09
Ocasionales    | compras por mes [333, 333, 334] | mañana 9.1% | ticket Q27.11
Validación correcta: integridad, importes, canjes, saldos y patrones de comportamiento.
```

5. Errores controlados y exportación Bronze

Agregamos 16 duplicados exactos y dos compras adicionales: una con importe negativo y otra con una referencia a un producto inexistente. Cada compra adicional contiene una línea asociada con un identificador nuevo. Así disponemos de ejemplos concretos para evaluar la detección de anomalías y la integridad de cada factura durante la limpieza.

Exportamos los CSV en UTF-8, con encabezados consistentes, decimales con punto y booleanos `true`/`false`. Cada ejecución actualiza los cinco archivos de Bronze.

Código · celda 12

```
bronze_tables = {name: [dict(row) for row in rows] for name, rows in clean_tables.items()}
duplicate_counts = {"clientes": 2, "productos": 2, "restaurantes": 2, "compras": 5, "lineas_compra": 5}
for name, count in duplicate_counts.items():
    bronze_tables[name].extend(dict(row) for row in clean_tables[name][:count])

negative_purchase_id = len(purchases) + 1
missing_product_purchase_id = len(purchases) + 2
negative_line_id = len(lines) + 1
missing_product_line_id = len(lines) + 2
for purchase_id, line_id, amount, product_id in [
    (negative_purchase_id, negative_line_id, "-12.35", 13),
    (missing_product_purchase_id, missing_product_line_id, "15.00", 999999),
]:
    bronze_tables["compras"].append({
        "id": purchase_id, "cliente_id": 1, "restaurante_id": 1,
        "fecha_hora": f"{YEAR}-03-31 22:59:59", "monto_total": amount, "forma_pago": "efectivo",
    })
    bronze_tables["lineas_compra"].append({
        "id": line_id, "compra_id": purchase_id, "producto_id": product_id, "cantidad": 1,
        "precio_unitario": amount, "subtotal": amount, "pagado_con_puntos": False,
    })

BRONZE.mkdir(parents=True, exist_ok=True)
for name, rows in bronze_tables.items():
    destination = BRONZE / f"{name}.csv"
    with destination.open("w", encoding="utf-8", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=SCHEMAS[name])
        writer.writeheader()
        for row in rows:
            writer.writerow({key: str(value).lower() if isinstance(value, bool) else value for key, value in row.items()})
    print(f"{destination.name:20s}: {len(rows):6,d} filas")
```

```
clientes.csv        :  1,002 filas
productos.csv       :     30 filas
restaurantes.csv    :     12 filas

compras.csv         : 20,007 filas
```

```
lineas_compra.csv   : 49,532 filas
```

6. Auditoría de los archivos exportados

Releemos los CSV y comprobamos sus tipos representados, conteos, duplicados y errores intencionales. Reconstruimos en memoria la base válida y revisamos el saldo histórico para verificar que la exportación conserva la información del escenario simulado.

Código · celda 14

```
integer_columns = {
    "clientes": {"id"}, "productos": {"id", "precio_en_puntos"}, "restaurantes": {"id"},
    "compras": {"id", "cliente_id", "restaurante_id"},
    "lineas_compra": {"id", "compra_id", "producto_id", "cantidad"},
}
loaded_tables = {}
for name in SCHEMAS:
    path = BRONZE / f"{name}.csv"
    with path.open(encoding="utf-8", newline="") as stream:
        reader = csv.DictReader(stream)
        assert reader.fieldnames == SCHEMAS[name]
        rows = list(reader)
    for row in rows:
        assert all(value is not None and value != "" for value in row.values())
        for column in integer_columns[name]:
            row[column] = int(row[column])
        if name == "lineas_compra":
            assert row["pagado_con_puntos"] in {"true", "false"}
            row["pagado_con_puntos"] = row["pagado_con_puntos"] == "true"
    loaded_tables[name] = rows
    assert rows == bronze_tables[name], f"La exportación alteró {name}"
    duplicates = len(rows) - len({tuple(row.values()) for row in rows})
    assert duplicates == duplicate_counts[name], (name, duplicates)
    extra_invalid = 2 if name in {"compras", "lineas_compra"} else 0
    assert len(rows) == len(clean_tables[name]) + duplicate_counts[name] + extra_invalid

negative_headers = [row["id"] for row in loaded_tables["compras"] if cents(row["monto_total"]) < 0]
negative_details = [row["id"] for row in loaded_tables["lineas_compra"] if cents(row["subtotal"]) < 0 or cents(row["precio_unitario"]) < 0]
unknown_products = [row["id"] for row in loaded_tables["lineas_compra"] if row["producto_id"] not in product_by_id]
assert negative_headers == [negative_purchase_id]
assert negative_details == [negative_line_id]
assert unknown_products == [missing_product_line_id]

# Reconstrucción de los originales conocidos para auditar el ciclo de escritura/lectura.
round_trip_clean = {}
for name, rows in loaded_tables.items():
    original_ids = {row["id"] for row in clean_tables[name]}
    round_trip_clean[name] = list({row["id"]: row for row in rows if row["id"] in original_ids}.values())
```

```
assert validate_clean_tables(round_trip_clean) == dict(balances)
print("Auditoría Bronze correcta: 16 duplicados y 2 compras con sus líneas inválidas.")
print(f"Saldo mínimo final bajo regla fija: {min(balances.values())} puntos.")
print("Huellas SHA-256 para comprobar reproducibilidad:")
for name in SCHEMAS:
    path = BRONZE / f"{name}.csv"
    print(f"  {path.name}: {hashlib.sha256(path.read_bytes()).hexdigest()}")

Auditoría Bronze correcta: 16 duplicados y 2 compras con sus líneas inválidas.
Saldo mínimo final bajo regla fija: 150 puntos.
Huellas SHA-256 para comprobar reproducibilidad:
  clientes.csv: 9db2f7cfb24bc221dfe8a30ff3bb9f949ba2960c48bb8099c6ce40a807f95841
  productos.csv: f1c4a4d9280f8c63c2dc0e9cf40974f885af1e5dc3e224e3a914b387de496acd
  restaurantes.csv: a976a4e05573cc3673268526e1f4396c4841ac1a363d728f3054330716008bab
  compras.csv: d2701a9b6f76ab974d90d526ab3700b5ecd296c94eaa9f0efb93b797e082b1e7
  lineas_compra.csv: dd4fb50b30a03d674b893b9adc3563b46f01dd47275f47e702cadc9eab00653e
```

Resultados

Obtenemos cinco fuentes sintéticas con transacciones coherentes, canjes financiados con saldo previo y patrones de compra útiles para la segmentación. Los errores controlados permiten evaluar la limpieza y el envío de registros a cuarentena.

Estos datos permiten comprobar el funcionamiento del sistema de puntos y estudiar la asignación de incentivos. La medición de su efecto comercial requiere datos reales y un piloto.

02 - Bronze → Silver

Limpiamos las cinco fuentes de Bronze con **DuckDB y SQL**. Organizamos los registros válidos en tablas Parquet y los rechazos en archivos CSV de cuarentena.

Criterios de limpieza:

- Normalizamos espacios y tipos, y conservamos los importes como `DECIMAL(12,2)`.
- Rechazamos campos obligatorios vacíos, tipos inválidos, valores fuera del dominio y referencias inexistentes.
- Conservamos una fila de cada duplicado exacto normalizado. Si un ID identifica contenidos distintos, rechazamos todas sus versiones.
- Rechazamos clientes distintos que comparten un correo normalizado.
- Validamos cada compra completa: una línea inválida, la ausencia de detalle, un producto repetido o un total inconsistente impiden aceptar la factura. Las copias duplicadas exactas se eliminan sin invalidar el original.

La cuarentena registra los valores originales, el archivo de origen, el número de registro CSV -incluido el encabezado- y los motivos de rechazo. Una fila puede presentar varios motivos.

Código · celda 02

```
from pathlib import Path
import csv
import hashlib
import duckdb

SCHEMAS = {
    "clientes": {"id": "INTEGER", "nombre": "VARCHAR", "correo": "VARCHAR", "fecha_registro": "DATE"},
    "productos": {"id": "INTEGER", "nombre": "VARCHAR", "categoria": "VARCHAR", "precio_normal": "DECIMAL(12,2)", "precio_en_puntos": "INTEGER"},
    "restaurantes": {"id": "INTEGER", "nombre": "VARCHAR", "zona": "VARCHAR"},
    "compras": {"id": "INTEGER", "cliente_id": "INTEGER", "restaurante_id": "INTEGER", "fecha_hora": "TIMESTAMP", "monto_total": "DECIMAL(12,2)", "forma_pago": "VARCHAR"},
    "lineas_compra": {"id": "INTEGER", "compra_id": "INTEGER", "producto_id": "INTEGER", "cantidad": "INTEGER", "precio_unitario": "DECIMAL(12,2)", "subtotal": "DECIMAL(12,2)",
     "pagado_con_puntos": "BOOLEAN"},
}
CATEGORIES = ('hamburguesas', 'pollo', 'desayunos', 'acompañamientos', 'bebidas', 'postres', 'ensaladas')


def sql_text(value):
    return "'" + str(value).replace("'", "''") + "'"


def typed_expression(column, data_type):
    value = f"nullif(trim({column}), '')"
    if data_type == "VARCHAR":
        return f"lower({value})" if column in {"correo", "categoria", "forma_pago"} else value
    if data_type == "INTEGER":
        valid = f"regexp_full_match({value}, '[+-]?[0-9]+')"
    elif data_type.startswith("DECIMAL"):
        # DuckDB puede redondear al convertir; esta validación evita perder precisión.
        valid = f"regexp_full_match({value}, '[+-]?[0-9]+([.][0-9]{{1,2}})?')"
    elif data_type == "BOOLEAN":
        valid = f"lower({value}) IN ('true', 'false')"
    elif data_type == "DATE":
        valid = f"regexp_full_match({value}, '[0-9]{{4}}-[0-9]{{2}}-[0-9]{{2}}')"
    else:
        valid = f"regexp_full_match({value}, '[0-9]{{4}}-[0-9]{{2}}-[0-9]{{2}}[ T][0-9]{{2}}:[0-9]{{2}}:[0-9]{{2}}([.][0-9]{{1,6}})?')"
    return f"CASE WHEN {valid} THEN try_cast({value} AS {data_type}) END"


def reject(connection, table, reason, condition, only_candidates=True):
    source = f"candidatos_{table}" if only_candidates else f"tipos_{table}"
    connection.execute(f"""
        INSERT INTO rechazos_{table}
        SELECT t._fila_csv, ? FROM {source} t WHERE {condition}
```

```
    """, [reason])


def load_and_type(connection, bronze):
    for table, schema in SCHEMAS.items():
        path = Path(bronze) / f"{table}.csv"
        if not path.is_file():
            raise FileNotFoundError(f"No se encuentra la fuente de datos: {path}.")
        with path.open(encoding="utf-8", newline="") as stream:
            header = next(csv.reader(stream), None)
        if header != list(schema):
            raise ValueError(f"Encabezado inesperado en {path.name}: {header}")
        # Una sola hebra y lectura secuencial mantienen el orden de los registros.
        connection.execute(f"""
            CREATE TABLE bronze_{table} AS
            SELECT row_number() OVER () + 1 AS _fila_csv, *
            FROM read_csv(?, header=true, all_varchar=true, parallel=false)
        """, [str(path)])
        columns = ', '.join(f"{typed_expression(name, dtype)} AS {name}" for name, dtype in schema.items())
        connection.execute(f"CREATE TABLE tipos_{table} AS SELECT _fila_csv, {columns} FROM bronze_{table}")
        connection.execute(f"CREATE TABLE rechazos_{table} (_fila_csv BIGINT, motivo VARCHAR)")
        connection.execute(f"""
            CREATE VIEW candidatos_{table} AS
            SELECT t.* FROM tipos_{table} t
            WHERE NOT EXISTS (SELECT 1 FROM rechazos_{table} r WHERE r._fila_csv = t._fila_csv)
        """)
        for column in schema:
            empty = f"t._fila_csv IN (SELECT _fila_csv FROM bronze_{table} WHERE nullif(trim({column}), '') IS NULL)"
            reject(connection, table, f"campo_obligatorio_vacio:{column}", empty, False)
            reject(connection, table, f"tipo_invalido:{column}", f"t.{column} IS NULL AND NOT ({empty})", False)
        for column, dtype in schema.items():
            if dtype == "INTEGER":
                reject(connection, table, f"entero_no_positivo:{column}", f"t.{column} <= 0", False)


def deduplicate(connection, table):
    fields = ', '.join(SCHEMAS[table])
    reject(connection, table, "id_conflictivo", f"""t.id IN (
        SELECT id FROM (SELECT DISTINCT {fields} FROM candidatos_{table})
        GROUP BY id HAVING count(*) > 1
    )""")
    reject(connection, table, "duplicado_exacto", f"""t._fila_csv IN (
        SELECT _fila_csv FROM (
            SELECT _fila_csv, row_number() OVER (PARTITION BY id ORDER BY _fila_csv) AS posicion
            FROM candidatos_{table}
        ) WHERE posicion > 1
    )""")
```

Reglas de negocio e integridad

Primero validamos los valores y resolvemos los duplicados. Después revisamos las relaciones en el orden clientes/productos/restaurantes → compras → líneas. Finalmente conciliamos las facturas completas y propagamos los rechazos a sus detalles.

Los canjes requieren precio y subtotal cero. Las líneas monetarias tienen un precio positivo y cumplen `subtotal = cantidad × precio_unitario`. Validamos el precio cobrado en la compra, que puede diferir del catálogo actual. Una compra compuesta únicamente por canjes admite un total de cero.

Revisamos que las fechas sean válidas, posteriores al registro del cliente y que las compras ocurran dentro del horario `[06:00, 23:00)`. La limpieza abarca todo el período de datos.

Código · celda 04

```
def clean_bronze(bronze):
    connection = duckdb.connect(":memory:")
    connection.execute("SET threads = 1")
    try:
        load_and_type(connection, bronze)
        category_values = ', '.join(sql_text(category) for category in CATEGORIES)
        rules = {
            "productos": [
                ("categoria_invalida", f"t.categoria NOT IN ({category_values})"),
                ("precio_normal_no_positivo", "t.precio_normal <= 0"),
            ],
            "compras": [
                ("monto_total_negativo", "t.monto_total < 0"),
                ("forma_pago_invalida", "t.forma_pago NOT IN ('efectivo', 'tarjeta')"),
                ("hora_fuera_de_rango", "cast(t.fecha_hora AS TIME) < TIME '06:00:00' OR cast(t.fecha_hora AS TIME) >= TIME '23:00:00'"),
            ],
            "lineas_compra": [
                ("precio_unitario_negativo", "t.precio_unitario < 0"),
                ("subtotal_negativo", "t.subtotal < 0"),
                ("subtotal_inconsistente", "t.subtotal != t.cantidad::BIGINT * t.precio_unitario"),
                ("canje_con_importe_monetario", "t.pagado_con_puntos AND (t.precio_unitario != 0 OR t.subtotal != 0)"),
                ("linea_monetaria_sin_precio", "NOT t.pagado_con_puntos AND t.precio_unitario <= 0"),
            ],
        }
        for table, checks in rules.items():
            for reason, condition in checks:
                reject(connection, table, reason, condition, False)
        for table in SCHEMAS:
            deduplicate(connection, table)
        reject(connection, "clientes", "correo_compartido", """t.correo IN (
            SELECT correo FROM candidatos_clientes GROUP BY correo HAVING count(*) > 1
        )""")
        reject(connection, "compras", "cliente_inexistente_o_rechazado", "t.cliente_id NOT IN (SELECT id FROM candidatos_clientes)")
```

```
        reject(connection, "compras", "restaurante_inexistente_o_rechazado", "t.restaurante_id NOT IN (SELECT id FROM candidatos_restaurantes)")
        reject(connection, "compras", "compra_anterior_al_registro", """EXISTS (
            SELECT 1 FROM candidatos_clientes c
            WHERE c.id = t.cliente_id AND cast(t.fecha_hora AS DATE) <= c.fecha_registro
        )""")
        reject(connection, "lineas_compra", "compra_inexistente_o_rechazada", "t.compra_id NOT IN (SELECT id FROM candidatos_compras)")
        reject(connection, "lineas_compra", "producto_inexistente_o_rechazado", "t.producto_id NOT IN (SELECT id FROM candidatos_productos)")
        # Una línea rechazada invalida la factura completa, salvo una copia exacta.
        reject(connection, "compras", "contiene_linea_rechazada", """EXISTS (
            SELECT 1 FROM tipos_lineas_compra l JOIN rechazos_lineas_compra r USING (_fila_csv)
            WHERE l.compra_id = t.id AND r.motivo != 'duplicado_exacto'
        )""")
        reject(connection, "compras", "sin_lineas_validas", "NOT EXISTS (SELECT 1 FROM candidatos_lineas_compra l WHERE l.compra_id = t.id)")
        reject(connection, "compras", "detalle_fuera_de_granularidad", """t.id IN (
            SELECT compra_id FROM candidatos_lineas_compra GROUP BY compra_id
            HAVING count(*) > 4 OR count(*) != count(DISTINCT producto_id)
        )""")
        reject(connection, "compras", "monto_total_inconsistente", """t.monto_total != (
            SELECT sum(l.subtotal) FROM candidatos_lineas_compra l WHERE l.compra_id = t.id
        )""")
        reject(connection, "lineas_compra", "compra_inexistente_o_rechazada", "t.compra_id NOT IN (SELECT id FROM candidatos_compras)")
        for table, schema in SCHEMAS.items():
            fields = ', '.join(schema)
            connection.execute(f"CREATE TABLE silver_{table} AS SELECT {fields} FROM candidatos_{table} ORDER BY id")
        return connection
    except Exception:
        connection.close()
        raise


def quality_summary(connection):
    rows = []
    for table in SCHEMAS:
        total = connection.execute(f"SELECT count(*) FROM bronze_{table}").fetchone()[0]
        accepted = connection.execute(f"SELECT count(*) FROM silver_{table}").fetchone()[0]
        rejected = connection.execute(f"SELECT count(DISTINCT _fila_csv) FROM rechazos_{table}").fetchone()[0]
        assert total == accepted + rejected, f"Filas sin clasificar o contadas dos veces: {table}"
        rows.append({"tabla": table, "bronze": total, "silver": accepted, "cuarentena": rejected})
    return rows


def validate_silver(connection):
    for table, schema in SCHEMAS.items():
        null_checks = ' OR '.join(f"{name} IS NULL" for name in schema)
        assert connection.execute(f"SELECT count(*) FROM silver_{table} WHERE {null_checks}").fetchone()[0] == 0
        assert connection.execute(f"SELECT count(*) - count(DISTINCT id) FROM silver_{table}").fetchone()[0] == 0
        actual = {row[0]: row[1] for row in connection.execute(f"DESCRIBE silver_{table}").fetchall()}
        assert actual == schema, (table, actual)
```

```
    for child, column, parent in [
        ('compras', 'cliente_id', 'clientes'), ('compras', 'restaurante_id', 'restaurantes'),
        ('lineas_compra', 'compra_id', 'compras'), ('lineas_compra', 'producto_id', 'productos'),
    ]:
        assert connection.execute(f"SELECT count(*) FROM silver_{child} WHERE {column} NOT IN (SELECT id FROM silver_{parent})").fetchone()[0] == 0
    assert connection.execute("SELECT count(*) - count(DISTINCT correo) FROM silver_clientes").fetchone()[0] == 0
    assert connection.execute("""
        SELECT count(*) FROM silver_compras c
        LEFT JOIN (SELECT compra_id, count(*) AS n, count(DISTINCT producto_id) AS productos,
                          sum(subtotal) AS total FROM silver_lineas_compra GROUP BY compra_id) l
        ON c.id = l.compra_id
        WHERE l.compra_id IS NULL OR l.n NOT BETWEEN 1 AND 4 OR l.productos != l.n OR l.total != c.monto_total
    """).fetchone()[0] == 0
    assert connection.execute("""
        SELECT count(*) FROM silver_lineas_compra
        WHERE cantidad < 1 OR precio_unitario < 0 OR subtotal < 0
           OR subtotal != cantidad::BIGINT * precio_unitario
           OR (pagado_con_puntos AND (precio_unitario != 0 OR subtotal != 0))
           OR (NOT pagado_con_puntos AND precio_unitario <= 0)
    """).fetchone()[0] == 0
    return quality_summary(connection)


def export_silver(connection, silver_dir, quarantine_dir):
    validate_silver(connection)
    silver_dir, quarantine_dir = Path(silver_dir), Path(quarantine_dir)
    silver_dir.mkdir(parents=True, exist_ok=True)
    quarantine_dir.mkdir(parents=True, exist_ok=True)
    for table, schema in SCHEMAS.items():
        parquet = silver_dir / f"{table}.parquet"
        quarantine = quarantine_dir / f"{table}.csv"
        connection.execute(f"COPY (SELECT * FROM silver_{table} ORDER BY id) TO {sql_text(parquet)} (FORMAT PARQUET, COMPRESSION ZSTD)")
        raw_columns = ', '.join(f"b.{name}" for name in schema)
        connection.execute(f"""
            COPY (
                SELECT {sql_text(table + '.csv')} AS archivo_origen, b._fila_csv AS fila_csv,
                       {raw_columns}, r.motivos
                FROM bronze_{table} b
                JOIN (SELECT _fila_csv, string_agg(DISTINCT motivo, ' | ' ORDER BY motivo) AS motivos
                      FROM rechazos_{table} GROUP BY _fila_csv) r USING (_fila_csv)
                ORDER BY b._fila_csv
            ) TO {sql_text(quarantine)} (FORMAT CSV, HEADER true)
        """)
        # Verificar contenido y tipos después de leer el Parquet escrito.
        connection.execute(f"CREATE OR REPLACE VIEW exportado_{table} AS SELECT * FROM read_parquet({sql_text(parquet)})")
        types = {row[0]: row[1] for row in connection.execute(f"DESCRIBE exportado_{table}").fetchall()}
        assert types == schema
        difference = connection.execute(f"""
```

```
            SELECT count(*) FROM (
                (SELECT * FROM silver_{table} EXCEPT ALL SELECT * FROM exportado_{table})
                UNION ALL
                (SELECT * FROM exportado_{table} EXCEPT ALL SELECT * FROM silver_{table})
            )
        """).fetchone()[0]
        assert difference == 0, f"El Parquet cambió los datos: {table}"
        with quarantine.open(encoding="utf-8", newline="") as stream:
            rows = list(csv.DictReader(stream))
        expected = connection.execute(f"SELECT count(DISTINCT _fila_csv) FROM rechazos_{table}").fetchone()[0]
        assert len(rows) == expected and all(row['motivos'] for row in rows)
```

Limpieza y revisión de resultados

Ejecutamos las reglas en memoria y obtenemos un resumen de registros aceptados y rechazados por tabla. Recalculamos la clasificación en cada ejecución para evitar la acumulación de rechazos de procesos anteriores.

Código · celda 06

```
PROJECT_ROOT = next((p for p in (Path.cwd(), *Path.cwd().parents) if (p / "doc.md").is_file()), None)
if PROJECT_ROOT is None:
    raise RuntimeError("No se encuentra la carpeta del caso. Inicie la ejecución desde Caso-Estudio-2.")
BRONZE = PROJECT_ROOT / "data" / "bronze"
SILVER = PROJECT_ROOT / "data" / "silver"
QUARANTINE = PROJECT_ROOT / "data" / "cuarentena"

def bronze_hashes():
    return {table: hashlib.sha256((BRONZE / f"{table}.csv").read_bytes()).hexdigest() for table in SCHEMAS}

before = bronze_hashes()
if "connection" in globals():
    connection.close()
connection = clean_bronze(BRONZE)
summary = validate_silver(connection)
print(f"DuckDB {duckdb.__version__}")
print(f"{'Tabla':20s} {'Bronze':>8s} {'Silver':>8s} {'Cuarentena':>12s}")
for row in summary:
    print(f"{row['tabla']:20s} {row['bronze']:8,d} {row['silver']:8,d} {row['cuarentena']:12,d}")
print("Todas las filas están contabilizadas: Bronze = Silver + cuarentena.")
```

```
DuckDB 1.5.5
Tabla                  Bronze   Silver   Cuarentena
clientes                1,002    1,000            2
productos                  30       28            2
restaurantes               12       10            2
compras                20,007   20,000            7
lineas_compra          49,532   49,525            7
Todas las filas están contabilizadas: Bronze = Silver + cuarentena.
```

Código · celda 07

```
# Conteos por motivo: una fila puede tener más de un motivo.
for table in SCHEMAS:
    reasons = connection.execute(f"SELECT motivo, count(DISTINCT _fila_csv) AS filas FROM rechazos_{table} GROUP BY motivo ORDER BY motivo").fetchall()
    print(f"\n{table}:")
    for reason, count in reasons:
        print(f"  {reason}: {count}")
```

```
clientes:
  duplicado_exacto: 2

productos:
  duplicado_exacto: 2

restaurantes:
  duplicado_exacto: 2

compras:
  contiene_linea_rechazada: 1
  duplicado_exacto: 5
  monto_total_negativo: 1

lineas_compra:
  duplicado_exacto: 5
  linea_monetaria_sin_precio: 1
  precio_unitario_negativo: 1
  producto_inexistente_o_rechazado: 1
  subtotal_negativo: 1
```

Exportación y verificación

Guardamos las cinco tablas válidas en `data/silver/` y los cinco archivos de rechazos en `data/cuarentena/`. Los CSV de cuarentena mantienen su encabezado incluso cuando no contienen rechazos.

Releemos los Parquet y comprobamos que fechas, enteros, decimales y booleanos conserven sus tipos y valores. También verificamos las claves, las relaciones y los montos.

Código · celda 09

```
export_silver(connection, SILVER, QUARANTINE)
assert before == bronze_hashes(), "Bronze fue modificado"
for table in SCHEMAS:
    print(f"{table}: Parquet y cuarentena verificados.")
print("Limpieza y validación de las tablas Silver completas.")
connection.close()
```

```
clientes: Parquet y cuarentena verificados.
productos: Parquet y cuarentena verificados.
```

```
restaurantes: Parquet y cuarentena verificados.
compras: Parquet y cuarentena verificados.
lineas_compra: Parquet y cuarentena verificados.
Limpieza y validación de las tablas Silver completas.
```

Resultados

Tabla	Bronze	Silver	Cuarentena
clientes	1,002	1,000	2
productos	30	28	2
restaurantes	12	10	2
compras	20,007	20,000	7
lineas_compra	49,532	49,525	7

Identificamos 20 registros rechazados: 16 duplicados, una compra negativa con su línea y una compra con producto inexistente junto con su detalle. Rechazamos esta última factura completa porque su línea no es válida.

Los conteos corresponden al escenario de la semilla actual. Las tablas limpias proporcionan la base para calcular los perfiles de clientes y el desempeño de los productos.

03 - Silver → Gold

Calculamos las tablas `perfil_clientes` y `ventas_productos` con **DuckDB** a partir de las compras de los dos primeros meses. Estos agregados describen el comportamiento de los clientes y el desempeño de los productos antes de aplicar los incentivos.

Criterios de análisis:

- Definimos la ventana como los dos meses calendario consecutivos desde el primer mes de Silver.
- Usamos el último día del segundo mes como referencia para la recencia.
- Identificamos la franja preferida por número de compras; resolvemos empates en el orden mañana, almuerzo, tarde y noche.
- Identificamos la categoría preferida por unidades compradas, incluidos canjes; resolvemos empates por orden alfabético.

- Calculamos las unidades vendidas y los ingresos únicamente con líneas monetarias.
- Consideramos rezagado un producto cuando `indice_ventas < 0.5`.

Los perfiles alimentan la segmentación, mientras las ventas permiten identificar los productos que necesitan un incentivo.

Código · celda 02

```
from pathlib import Path
import duckdb

PROJECT_ROOT = next((p for p in (Path.cwd(), *Path.cwd().parents) if (p / "doc.md").is_file()), None)
if PROJECT_ROOT is None:
    raise RuntimeError("No se encuentra la carpeta del caso. Inicie la ejecución desde Caso-Estudio-2.")
SILVER = PROJECT_ROOT / "data" / "silver"
GOLD = PROJECT_ROOT / "data" / "gold"

MESES_VENTANA = 2
UMBRAL_REZAGO = 0.5
FRANJAS = ["mañana", "almuerzo", "tarde", "noche"]


def sql_text(value):
    return "'" + str(value).replace("'", "''") + "'"


connection = duckdb.connect(":memory:")
connection.execute("SET threads = 1")
for table in ("clientes", "productos", "restaurantes", "compras", "lineas_compra"):
    parquet = sql_text(SILVER / f"{table}.parquet")
    connection.execute(f"CREATE VIEW {table} AS SELECT * FROM read_parquet({parquet})")
print(f"DuckDB {duckdb.__version__}")
```

```
DuckDB 1.5.5
```

Ventana de análisis

Tomamos los dos meses calendario consecutivos desde el primer mes de Silver. Fijamos la fecha de referencia para la recencia en el último día del segundo mes, justo antes del inicio de la aplicación de incentivos.

Código · celda 04

```
inicio_mes1, fin_ventana, fecha_referencia = connection.execute("""
    SELECT
        date_trunc('month', min(fecha_hora)) AS inicio_mes1,
        date_trunc('month', min(fecha_hora)) + INTERVAL 2 MONTH AS fin_ventana,
        date_trunc('month', min(fecha_hora)) + INTERVAL 2 MONTH - INTERVAL 1 DAY AS fecha_referencia
    FROM compras
```

```
""").fetchone()

connection.execute(f"""
    CREATE VIEW ventana AS
    SELECT * FROM compras
    WHERE fecha_hora >= {sql_text(inicio_mes1)}::TIMESTAMP
      AND fecha_hora < {sql_text(fin_ventana)}::TIMESTAMP
""")
print(f"Ventana: [{inicio_mes1.date()}, {fin_ventana.date()}) - fecha de referencia: {fecha_referencia.date()}")
```

```
Ventana: [2025-01-01, 2025-03-01) - fecha de referencia: 2025-02-28
```

perfil_clientes

Resumimos el comportamiento de cada cliente en una fila: recencia, frecuencia mensual, gasto promedio, franja horaria preferida y categoría preferida. Utilizamos estas variables como base para identificar grupos con hábitos de compra similares.

Código · celda 06

```
franjas_sql = "[" + ", ".join(sql_text(franja) for franja in FRANJAS) + "]"

perfil_clientes = connection.execute(f"""
    WITH franja_compra AS (
        SELECT *,
            CASE
                WHEN extract(hour FROM fecha_hora) BETWEEN 6 AND 10 THEN 'mañana'
                WHEN extract(hour FROM fecha_hora) BETWEEN 11 AND 14 THEN 'almuerzo'
                WHEN extract(hour FROM fecha_hora) BETWEEN 15 AND 17 THEN 'tarde'
                ELSE 'noche'
            END AS franja
        FROM ventana
    ),
    conteo_franja AS (
        SELECT cliente_id, franja, count(*) AS n
        FROM franja_compra
        GROUP BY 1, 2
    ),
    franja_preferida AS (
        SELECT cliente_id, franja AS franja_preferida
        FROM (
            SELECT cliente_id, franja,
                   row_number() OVER (
                       PARTITION BY cliente_id
                       ORDER BY n DESC, list_position({franjas_sql}, franja)
                   ) AS posicion
            FROM conteo_franja
        )
```

```
        WHERE posicion = 1
    ),
    unidades_categoria_cliente AS (
        SELECT v.cliente_id, p.categoria, sum(l.cantidad) AS unidades
        FROM ventana v
        JOIN lineas_compra l ON l.compra_id = v.id
        JOIN productos p ON p.id = l.producto_id
        GROUP BY 1, 2
    ),
    categoria_preferida AS (
        SELECT cliente_id, categoria AS categoria_preferida
        FROM (
            SELECT cliente_id, categoria,
                   row_number() OVER (PARTITION BY cliente_id ORDER BY unidades DESC, categoria) AS posicion
            FROM unidades_categoria_cliente
        )
        WHERE posicion = 1
    ),
    resumen AS (
        SELECT cliente_id,
               count(*) AS num_compras,
               avg(monto_total) AS gasto_promedio,
               date_diff('day', max(fecha_hora)::DATE, {sql_text(fecha_referencia)}::DATE) AS recencia_dias
        FROM ventana
        GROUP BY 1
    )
    SELECT
        c.id AS cliente_id,
        r.recencia_dias,
        round(r.num_compras / {MESES_VENTANA}.0, 2) AS frecuencia_mensual,
        round(r.gasto_promedio, 2) AS gasto_promedio,
        fp.franja_preferida,
        cp.categoria_preferida
    FROM clientes c
    LEFT JOIN resumen r ON r.cliente_id = c.id
    LEFT JOIN franja_preferida fp ON fp.cliente_id = c.id
    LEFT JOIN categoria_preferida cp ON cp.cliente_id = c.id
    ORDER BY c.id
""").df()
perfil_clientes.head()
```

	cliente_id	recencia_dias	frecuencia_mensual	gasto_promedio	franja_preferida	categoria_preferida
0	1	17	1.0	23.75	almuerzo	acompañamientos
1	2	2	14.0	48.80	mañana	desayunos

	cliente_id	recencia_dias	frecuencia_mensual	gasto_promedio	franja_preferida	categoria_preferida
2	3	1	18.5	99.88	noche	hamburguesas
3	4	2	2.0	21.98	almuerzo	acompañamientos
4	5	6	1.0	57.65	tarde	bebidas

ventas_productos

Resumimos las unidades vendidas y los ingresos de cada producto y comparamos su desempeño con el promedio de su categoría. Contamos únicamente líneas monetarias, ya que los canjes representan uso de recompensas y no generan ingresos en la compra.

Código · celda 08

```
ventas_productos = connection.execute(f"""
    WITH lineas_dinero AS (
        SELECT l.producto_id, l.cantidad, l.subtotal
        FROM ventana v
        JOIN lineas_compra l ON l.compra_id = v.id
        WHERE NOT l.pagado_con_puntos
    ),
    ventas_producto AS (
        SELECT producto_id,
               sum(cantidad) AS unidades_vendidas,
               sum(subtotal) AS ingresos
        FROM lineas_dinero
        GROUP BY 1
    ),
    ventas_por_producto AS (
        SELECT p.id AS producto_id, p.categoria,
               coalesce(v.unidades_vendidas, 0)::BIGINT AS unidades_vendidas,
               coalesce(v.ingresos, 0) AS ingresos
        FROM productos p
        LEFT JOIN ventas_producto v ON v.producto_id = p.id
    ),
    promedio_por_categoria AS (
        SELECT categoria, avg(unidades_vendidas) AS promedio_categoria
        FROM ventas_por_producto
        GROUP BY 1
    )
    SELECT
        vp.producto_id,
        vp.categoria,
```

```
        vp.unidades_vendidas,
        round(vp.ingresos, 2) AS ingresos,
        round(pc.promedio_categoria, 4) AS promedio_categoria,
        round(vp.unidades_vendidas / pc.promedio_categoria, 4) AS indice_ventas,
        (vp.unidades_vendidas / pc.promedio_categoria) < {UMBRAL_REZAGO} AS es_rezagado
    FROM ventas_por_producto vp
    JOIN promedio_por_categoria pc ON pc.categoria = vp.categoria
    ORDER BY vp.producto_id
""").df()
ventas_productos.head()
```

	producto_id	categoria	unidades_vendidas	ingresos	promedio_categoria	indice_ventas	es_rezagado
0	1	hamburguesas	2959	106376.05	2235.0	1.3239	False
1	2	hamburguesas	2893	112537.70	2235.0	1.2944	False
2	3	hamburguesas	2946	53028.00	2235.0	1.3181	False
3	4	hamburguesas	142	4615.00	2235.0	0.0635	True
4	5	pollo	2472	79104.00	1823.5	1.3556	False

Validación y exportación

Comprobamos que cada cliente y producto tenga una sola fila, que las variables requeridas estén completas y que la clasificación de rezago coincida con el umbral establecido. Guardamos los perfiles y las ventas en Parquet dentro de la capa Gold.

Código · celda 10

```
num_clientes = connection.execute("SELECT count(*) FROM clientes").fetchone()[0]
num_productos = connection.execute("SELECT count(*) FROM productos").fetchone()[0]

assert len(perfil_clientes) == num_clientes, "Cada cliente debe tener una fila en perfil_clientes"
assert perfil_clientes.isna().sum().sum() == 0, "perfil_clientes no debe tener nulos en la ventana analizada"
assert perfil_clientes["recencia_dias"].min() >= 0, "La recencia no puede ser negativa"
assert perfil_clientes["franja_preferida"].isin(FRANJAS).all(), "Franja fuera de las cuatro válidas"

assert len(ventas_productos) == num_productos, "Cada producto debe tener una fila en ventas_productos"
assert (ventas_productos["unidades_vendidas"] >= 0).all(), "Las unidades vendidas no pueden ser negativas"
assert ventas_productos["indice_ventas"].notna().all(), "indice_ventas no debe ser nulo"
assert (ventas_productos["es_rezagado"] == (ventas_productos["indice_ventas"] < UMBRAL_REZAGO)).all(), "es_rezagado no coincide con el umbral"

GOLD.mkdir(parents=True, exist_ok=True)
```

```
connection.register("perfil_clientes_df", perfil_clientes)
connection.register("ventas_productos_df", ventas_productos)
connection.execute(f"COPY (SELECT * FROM perfil_clientes_df ORDER BY cliente_id) TO {sql_text(GOLD / 'perfil_clientes.parquet')} (FORMAT PARQUET, COMPRESSION ZSTD)")
connection.execute(f"COPY (SELECT * FROM ventas_productos_df ORDER BY producto_id) TO {sql_text(GOLD / 'ventas_productos.parquet')} (FORMAT PARQUET, COMPRESSION ZSTD)")

for name, esperado in (("perfil_clientes", num_clientes), ("ventas_productos", num_productos)):
    filas = connection.execute(f"SELECT count(*) FROM read_parquet({sql_text(GOLD / f'{name}.parquet')})").fetchone()[0]
    assert filas == esperado, f"{name}: se exportaron {filas} filas, se esperaban {esperado}"
    print(f"{name}: {filas} filas exportadas a Gold.")

rezagados = int(ventas_productos["es_rezagado"].sum())
print(f"Productos rezagados (indice_ventas < {UMBRAL_REZAGO}): {rezagados} de {num_productos}.")
print("Distribución de franja_preferida:")
print(perfil_clientes["franja_preferida"].value_counts())

connection.close()
```

```
perfil_clientes: 1000 filas exportadas a Gold.
ventas_productos: 28 filas exportadas a Gold.
Productos rezagados (indice_ventas < 0.5): 4 de 28.
Distribución de franja_preferida:
franja_preferida
mañana      379
almuerzo    299
noche       166
tarde       156
Name: count, dtype: int64
```

Resultados

Obtenemos un perfil por cliente y un resumen de ventas por producto. Estos agregados permiten segmentar el comportamiento e identificar productos con baja rotación relativa.

Todos los clientes del escenario presentan compras en la ventana de análisis y todas las categorías registran ventas. Estas condiciones permiten calcular los indicadores sin imputar valores para clientes sin historial o categorías sin compras.

04 - Segmentación ML

Agrupamos a los clientes con **K-means** y calculamos la afinidad entre segmentos y categorías con **DuckDB**. Utilizamos los perfiles de Gold para identificar hábitos de compra y orientar la asignación de incentivos.

Criterios del modelo:

- Utilizamos recencia, frecuencia mensual, gasto promedio y franja preferida; transformamos la franja en columnas numéricas y estandarizamos las variables.
- Reservamos la categoría preferida para interpretar los grupos.
- Entrenamos con variables observables, sin las etiquetas ocultas de la generación sintética.
- Definimos cuatro segmentos y una semilla fija para reproducir el escenario.
- Calculamos la afinidad con las compras de los dos primeros meses, incluidos los canjes, porque también reflejan elección de categorías.
- Consideramos afinidad alta cuando `indice_afinidad > 1.2`.

Incorporamos el segmento al perfil de cada cliente y resumimos las afinidades en una tabla de Gold.

Código · celda 02

```
from pathlib import Path
import duckdb
import numpy as np
import pandas as pd
import sklearn
from sklearn.cluster import KMeans
from sklearn.compose import ColumnTransformer
from sklearn.metrics import adjusted_rand_score, silhouette_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

PROJECT_ROOT = next((p for p in (Path.cwd(), *Path.cwd().parents) if (p / "doc.md").is_file()), None)
if PROJECT_ROOT is None:
    raise RuntimeError("No se encuentra la carpeta del caso. Inicie la ejecución desde Caso-Estudio-2.")
SILVER = PROJECT_ROOT / "data" / "silver"
GOLD = PROJECT_ROOT / "data" / "gold"

SEED = 22434
NUM_SEGMENTOS = 4
UMBRAL_AFINIDAD = 1.2
FRANJAS = ["mañana", "almuerzo", "tarde", "noche"]
SEGMENTOS = ["Leales", "Madrugadores", "En riesgo", "Ocasionales"]
VARIABLES_NUMERICAS = ["recencia_dias", "frecuencia_mensual", "gasto_promedio"]
# Ponderar el bloque one-hot de franja como una sola variable.
```

```
PESO_FRANJA = 1 / np.sqrt(len(FRANJAS))


def sql_text(value):
    return "'" + str(value).replace("'", "''") + "'"


connection = duckdb.connect(":memory:")
connection.execute("SET threads = 1")
for table in ("clientes", "productos", "compras", "lineas_compra"):
    parquet = sql_text(SILVER / f"{table}.parquet")
    connection.execute(f"CREATE VIEW {table} AS SELECT * FROM read_parquet({parquet})")

# Al reejecutar, perfil_clientes ya trae un segmento previo: se descarta para recalcularlo.
perfil_clientes = connection.execute(
    f"SELECT * FROM read_parquet({sql_text(GOLD / 'perfil_clientes.parquet')}) ORDER BY cliente_id"
).df().drop(columns="segmento", errors="ignore")
print(f"DuckDB {duckdb.__version__} - scikit-learn {sklearn.__version__}")
print(f"perfil_clientes: {len(perfil_clientes)} clientes")
perfil_clientes.head()
```

```
DuckDB 1.5.5 - scikit-learn 1.9.1
perfil_clientes: 1000 clientes
```

	cliente_id	recencia_dias	frecuencia_mensual	gasto_promedio	franja_preferida	categoria_preferida
0	1	17	1.0	23.75	almuerzo	acompañamientos
1	2	2	14.0	48.80	mañana	desayunos
2	3	1	18.5	99.88	noche	hamburguesas
3	4	2	2.0	21.98	almuerzo	acompañamientos
4	5	6	1.0	57.65	tarde	bebidas

Preparación de variables

K-means utiliza distancias entre variables numéricas. Preparamos los datos en tres pasos:

1. Convertimos `franja_preferida` en cuatro columnas 0/1, una por franja.
2. Estandarizamos las columnas para que las diferencias de escala no dominen la agrupación.

3. Ponderamos el bloque de franja por `1/√4 = 0.5`. Así, sus cuatro columnas aportan en conjunto una unidad de varianza, comparable con cada variable numérica.

Esta preparación equilibra la contribución de recencia, frecuencia, gasto y horario.

Código · celda 04

```
preprocesamiento = ColumnTransformer(
    [
        ("numericas", StandardScaler(), VARIABLES_NUMERICAS),
        ("franja", Pipeline([
            ("one_hot", OneHotEncoder(categories=[FRANJAS], sparse_output=False)),
            ("escala", StandardScaler()),
        ]), ["franja_preferida"]),
    ],
    transformer_weights={"numericas": 1.0, "franja": PESO_FRANJA},
)
X = preprocesamiento.fit_transform(perfil_clientes)
columnas_modelo = list(preprocesamiento.get_feature_names_out())

assert not any("categoria" in columna for columna in columnas_modelo), "categoria_preferida no debe entrar al modelo"
assert X.shape == (len(perfil_clientes), len(VARIABLES_NUMERICAS) + len(FRANJAS))
assert not np.isnan(X).any(), "La matriz del modelo no debe tener nulos"

varianza = pd.Series(X.var(axis=0), index=columnas_modelo).round(3)
print("Varianza por columna del modelo:")
print(varianza.to_string())
print(f"Varianza total del bloque franja: {varianza.filter(like='franja').sum():.3f} (≈ una variable)")
```

```
Varianza por columna del modelo:
numericas__recencia_dias             1.00
numericas__frecuencia_mensual        1.00
numericas__gasto_promedio            1.00
franja__franja_preferida_mañana      0.25
franja__franja_preferida_almuerzo    0.25
franja__franja_preferida_tarde       0.25
franja__franja_preferida_noche       0.25
Varianza total del bloque franja: 1.000 (≈ una variable)
```

Diagnóstico del número de grupos

Planteamos cuatro segmentos por su utilidad para interpretar el comportamiento de compra. Evaluamos la inercia y el coeficiente de silueta para k = 2…8. La silueta varía entre -1 y 1; los valores mayores indican grupos más compactos y separados.

Con k = 4 obtenemos una silueta de 0.4280, mientras k = 6 alcanza 0.4499. Mantenemos cuatro grupos por la interpretación de negocio, aunque esta elección no maximiza la silueta.

Código · celda 06

```
diagnostico = []
for k in range(2, 9):
    modelo_k = KMeans(n_clusters=k, n_init=20, random_state=SEED).fit(X)
    diagnostico.append({
        "k": k,
        "inercia": round(modelo_k.inertia_, 1),
        "silueta": round(silhouette_score(X, modelo_k.labels_), 4),
    })
diagnostico = pd.DataFrame(diagnostico).set_index("k")
diagnostico
```

	inercia	silueta
k		
2	2562.6	0.3380
3	1768.0	0.3867
4	1265.2	0.4280
5	1122.1	0.4298
6	982.5	0.4499
7	878.6	0.4343
8	796.1	0.4482

Entrenamiento de K-means

Entrenamos cuatro grupos con 20 inicializaciones y seleccionamos la solución de menor inercia. Evaluamos la estabilidad con otras diez semillas y comparamos las particiones mediante el índice de Rand ajustado (ARI). Un valor de 1 indica los mismos grupos, aunque cambie su numeración.

Código · celda 08

```
kmeans = KMeans(n_clusters=NUM_SEGMENTOS, n_init=20, random_state=SEED)
grupos = kmeans.fit_predict(X)

estabilidad = [
    adjusted_rand_score(grupos, KMeans(n_clusters=NUM_SEGMENTOS, n_init=20, random_state=semilla).fit_predict(X))
    for semilla in range(1, 11)
```

```
]
silueta = silhouette_score(X, grupos)

assert len(set(grupos)) == NUM_SEGMENTOS, "K-means debe producir cuatro grupos no vacíos"
assert min(estabilidad) > 0.9, f"Segmentación inestable entre semillas (ARI mínimo {min(estabilidad):.3f})"
print(f"Silueta con k={NUM_SEGMENTOS}: {silueta:.4f}")
print(f"ARI frente a otras 10 semillas: mínimo {min(estabilidad):.4f}, promedio {np.mean(estabilidad):.4f}")
print("Clientes por grupo:", dict(sorted(pd.Series(grupos).value_counts().items())))
```

```
Silueta con k=4: 0.4280
ARI frente a otras 10 semillas: mínimo 1.0000, promedio 1.0000
Clientes por grupo: {0: 250, 1: 255, 2: 249, 3: 246}
```

Interpretación de los grupos

K-means asigna números a los grupos. Para interpretarlos, resumimos su tamaño, recencia, frecuencia, gasto, distribución de la franja preferida y categoría preferida más común. Expresamos los indicadores en sus unidades originales; la categoría apoya la interpretación y no participa en el entrenamiento.

Código · celda 10

```
perfil_grupos = perfil_clientes.assign(grupo=grupos)

resumen_grupos = perfil_grupos.groupby("grupo").agg(
    clientes=("cliente_id", "size"),
    recencia_dias=("recencia_dias", "mean"),
    frecuencia_mensual=("frecuencia_mensual", "mean"),
    gasto_promedio=("gasto_promedio", "mean"),
)
pct_franja = (
    pd.crosstab(perfil_grupos["grupo"], perfil_grupos["franja_preferida"], normalize="index")
    .reindex(columns=FRANJAS, fill_value=0)
    .add_prefix("pct_")
)
categoria_comun = perfil_grupos.groupby("grupo")["categoria_preferida"].agg(
    lambda categorias: categorias.value_counts().index[0]
).rename("categoria_mas_comun")

resumen_grupos = resumen_grupos.join(pct_franja).join(categoria_comun).round(2)
resumen_grupos
```

	clientes	recencia_dias	frecuencia_mensual	gasto_promedio	pct_mañana	pct_almuerzo	pct_tarde	pct_noche	categoria_mas_comun
grupo									
0	250	1.94	12.00	102.68	0.31	0.28	0.17	0.24	hamburguesas

	clientes	recencia_dias	frecuencia_mensual	gasto_promedio	pct_mañana	pct_almuerzo	pct_tarde	pct_noche	categoria_mas_comun
1	255	26.08	6.38	58.78	0.04	0.65	0.04	0.27	ensaladas
2	249	2.79	8.69	39.12	1.00	0.00	0.00	0.00	desayunos
3	246	11.25	1.36	26.41	0.17	0.26	0.42	0.15	acompañamientos

Asignación de nombres

Asignamos nombres mediante reglas explícitas sobre las características de cada grupo. Seguimos este orden para mantener una interpretación reproducible:

Orden	Segmento	Regla	Interpretación
1	Ocasionales	Menor `frecuencia_mensual`	Presentan pocas compras; su recencia también puede ser alta
2	Madrugadores	Mayor proporción de franja `mañana` entre los restantes	Concentran su actividad en la mañana
3	En riesgo	Mayor `recencia_dias` entre los dos restantes	Presentan frecuencia intermedia y menor actividad reciente
4	Leales	Grupo restante	Presentan mayor frecuencia y gasto

Verificamos que los indicadores de cada segmento coincidan con esta interpretación. Las comprobaciones detectan si un cambio en los datos produce grupos que requieren una interpretación distinta.

Código · celda 12

```
def nombrar_grupos(resumen):
    pendientes = set(resumen.index)

    def elegir(columna, criterio):
        valores = resumen.loc[sorted(pendientes), columna]
        grupo = valores.idxmin() if criterio == "min" else valores.idxmax()
        pendientes.remove(grupo)
        return grupo

    nombres = {}
    nombres[elegir("frecuencia_mensual", "min")] = "Ocasionales"
    nombres[elegir("pct_mañana", "max")] = "Madrugadores"
    nombres[elegir("recencia_dias", "max")] = "En riesgo"
```

```
    nombres[pendientes.pop()] = "Leales"
    return {int(grupo): nombre for grupo, nombre in nombres.items()}


nombres_grupo = nombrar_grupos(resumen_grupos)
resumen_segmentos = resumen_grupos.rename(index=nombres_grupo).rename_axis("segmento").loc[SEGMENTOS]

assert sorted(nombres_grupo.values()) == sorted(SEGMENTOS), "Cada segmento debe asignarse a un solo grupo"
leales, madrugadores, riesgo, ocasionales = (resumen_segmentos.loc[s] for s in SEGMENTOS)
assert leales["frecuencia_mensual"] == resumen_segmentos["frecuencia_mensual"].max(), "Leales: deben ser los más frecuentes"
assert leales["gasto_promedio"] == resumen_segmentos["gasto_promedio"].max(), "Leales: deben tener el mayor gasto"
assert madrugadores["pct_mañana"] >= 0.8, "Madrugadores: la mayoría debe preferir la mañana"
assert riesgo["recencia_dias"] > max(leales["recencia_dias"], madrugadores["recencia_dias"]), "En riesgo: menos recientes que los activos"
assert riesgo["frecuencia_mensual"] > ocasionales["frecuencia_mensual"], "En riesgo: compraban más que los Ocasionales"

perfil_clientes["segmento"] = pd.Series(grupos).map(nombres_grupo).to_numpy()
print("Grupo → segmento:", dict(sorted(nombres_grupo.items())))
resumen_segmentos
```

```
Grupo → segmento: {0: 'Leales', 1: 'En riesgo', 2: 'Madrugadores', 3: 'Ocasionales'}
```

	clientes	recencia_dias	frecuencia_mensual	gasto_promedio	pct_mañana	pct_almuerzo	pct_tarde	pct_noche	categoria_mas_comun
segmento									
Leales	250	1.94	12.00	102.68	0.31	0.28	0.17	0.24	hamburguesas
Madrugadores	249	2.79	8.69	39.12	1.00	0.00	0.00	0.00	desayunos
En riesgo	255	26.08	6.38	58.78	0.04	0.65	0.04	0.27	ensaladas
Ocasionales	246	11.25	1.36	26.41	0.17	0.26	0.42	0.15	acompañamientos

Examinamos el porcentaje de clientes de cada segmento que prefiere cada categoría. Esta distribución complementa la interpretación: los Madrugadores, por ejemplo, concentran sus preferencias en desayunos y bebidas.

Código · celda 14

```
(
    pd.crosstab(perfil_clientes["segmento"], perfil_clientes["categoria_preferida"], normalize="index")
    .loc[SEGMENTOS]
    .mul(100)
    .round(1)
```

)

categoria_preferida	acompañamientos	bebidas	desayunos	ensaladas	hamburguesas	pollo	postres
segmento							
Leales	0.0	0.0	0.0	0.0	69.6	26.0	4.4
Madrugadores	0.0	19.7	80.3	0.0	0.0	0.0	0.0
En riesgo	2.7	2.4	0.4	78.4	12.9	3.1	0.0
Ocasionales	49.2	42.7	1.2	0.8	1.6	0.8	3.7

afinidad_segmento_categoria

Calculamos una fila por segmento y categoría con las compras de los dos primeros meses:

- `pct_compras_segmento`: porcentaje de compras del segmento que incluyen al menos un producto de la categoría.
- `pct_compras_total`: porcentaje equivalente sobre todas las compras de la ventana.
- `indice_afinidad`: cociente entre ambos porcentajes. Un valor mayor que 1 indica una presencia de la categoría superior al promedio.

Calculamos el índice antes de redondear e incluimos las 28 combinaciones de cuatro segmentos y siete categorías.

Código · celda 16

```
inicio_mes1, fin_ventana = connection.execute("""
    SELECT date_trunc('month', min(fecha_hora)) AS inicio_mes1,
           date_trunc('month', min(fecha_hora)) + INTERVAL 2 MONTH AS fin_ventana
    FROM compras
""").fetchone()
print(f"Ventana: [{inicio_mes1.date()}, {fin_ventana.date()})")

segmentos_sql = "[" + ", ".join(sql_text(segmento) for segmento in SEGMENTOS) + "]"
connection.register("segmentos_df", perfil_clientes[["cliente_id", "segmento"]])

afinidad_segmento_categoria = connection.execute(f"""
    WITH compras_ventana AS (
        SELECT c.id AS compra_id, s.segmento
        FROM compras c
        JOIN segmentos_df s ON s.cliente_id = c.cliente_id
        WHERE c.fecha_hora >= {sql_text(inicio_mes1)}::TIMESTAMP
```

```
          AND c.fecha_hora < {sql_text(fin_ventana)}::TIMESTAMP
    ),
    compra_categoria AS (
        SELECT DISTINCT cv.compra_id, cv.segmento, p.categoria
        FROM compras_ventana cv
        JOIN lineas_compra l ON l.compra_id = cv.compra_id
        JOIN productos p ON p.id = l.producto_id
    ),
    compras_segmento AS (
        SELECT segmento, count(*)::DOUBLE AS compras FROM compras_ventana GROUP BY 1
    ),
    compras_total AS (
        SELECT count(*)::DOUBLE AS compras FROM compras_ventana
    ),
    categorias AS (
        SELECT DISTINCT categoria FROM productos
    ),
    categoria_segmento AS (
        SELECT segmento, categoria, count(*) AS compras FROM compra_categoria GROUP BY 1, 2
    ),
    categoria_total AS (
        SELECT categoria, count(*) AS compras FROM compra_categoria GROUP BY 1
    ),
    proporciones AS (
        SELECT
            cs.segmento,
            cat.categoria,
            coalesce(sc.compras, 0) / cs.compras AS prop_segmento,
            coalesce(ct.compras, 0) / t.compras AS prop_total
        FROM compras_segmento cs
        CROSS JOIN categorias cat
        CROSS JOIN compras_total t
        LEFT JOIN categoria_segmento sc ON sc.segmento = cs.segmento AND sc.categoria = cat.categoria
        LEFT JOIN categoria_total ct ON ct.categoria = cat.categoria
    )
    SELECT
        segmento,
        categoria,
        round(100 * prop_segmento, 2) AS pct_compras_segmento,
        round(100 * prop_total, 2) AS pct_compras_total,
        round(prop_segmento / prop_total, 4) AS indice_afinidad
    FROM proporciones
    ORDER BY list_position({segmentos_sql}, segmento), categoria
""").df()
afinidad_segmento_categoria.head(7)

Ventana: [2025-01-01, 2025-03-01)
```

	segmento	categoria	pct_compras_segmento	pct_compras_total	indice_afinidad
0	Leales	acompañamientos	31.75	21.22	1.4965
1	Leales	bebidas	23.12	32.44	0.7125
2	Leales	desayunos	14.02	31.22	0.4490
3	Leales	ensaladas	8.52	18.72	0.4550
4	Leales	hamburguesas	64.62	38.04	1.6984
5	Leales	pollo	56.40	32.30	1.7463
6	Leales	postres	43.02	24.86	1.7300

Comparamos la afinidad de cada segmento con el umbral de 1.2 y la relacionamos con las categorías de los productos rezagados. Este cruce permite identificar a qué clientes dirigir los incentivos de cada producto.

Código · celda 18

```
matriz_afinidad = afinidad_segmento_categoria.pivot(index="segmento", columns="categoria", values="indice_afinidad").loc[SEGMENTOS]
print(matriz_afinidad.round(2).to_string())

ventas_productos = connection.execute(
    f"SELECT * FROM read_parquet({sql_text(GOLD / 'ventas_productos.parquet')})"
).df()
rezagados = ventas_productos.loc[ventas_productos["es_rezagado"]].merge(
    connection.execute("SELECT id AS producto_id, nombre FROM productos").df(), on="producto_id"
)
print("\nProductos rezagados y segmentos con afinidad > umbral en su categoría:")
for producto in rezagados.itertuples():
    afines = afinidad_segmento_categoria[
        (afinidad_segmento_categoria["categoria"] == producto.categoria)
        & (afinidad_segmento_categoria["indice_afinidad"] > UMBRAL_AFINIDAD)
    ]
    destino = ", ".join(f"{fila.segmento} ({fila.indice_afinidad:.2f})" for fila in afines.itertuples()) or "ninguno → todos los clientes"
    print(f"  {producto.nombre} [{producto.categoria}]: {destino}")
```

```
categoria     acompañamientos  bebidas  desayunos  ensaladas  hamburguesas  pollo  postres
segmento
Leales                   1.50     0.71       0.45       0.46          1.70   1.75     1.73
Madrugadores             0.26     1.65       2.26       0.30          0.16   0.12     0.24
En riesgo                0.89     0.58       0.49       3.11          0.98   0.95     0.72
Ocasionales              1.88     1.43       0.26       0.12          0.28   0.20     0.77
```

```
Productos rezagados y segmentos con afinidad > umbral en su categoría:
  Hamburguesa especial [hamburguesas]: Leales (1.70)
  Sándwich de pollo especial [pollo]: Leales (1.75)
  Pie de manzana [postres]: Leales (1.73)
  Ensalada especial [ensaladas]: En riesgo (3.11)
```

Validación y exportación

Comprobamos que cada cliente tenga un único segmento válido y que la tabla de afinidad contenga una fila por segmento y categoría, sin valores nulos ni negativos.

También verificamos que el porcentaje total de compras por categoría coincida con el promedio de los porcentajes de cada segmento ponderado por su número de compras, con una tolerancia para el redondeo.

Guardamos en Gold el perfil con su segmento y la tabla de afinidad.

Código · celda 20

```
num_clientes = connection.execute("SELECT count(*) FROM clientes").fetchone()[0]
num_categorias = connection.execute("SELECT count(DISTINCT categoria) FROM productos").fetchone()[0]

assert len(perfil_clientes) == num_clientes, "Cada cliente debe tener una fila en perfil_clientes"
assert perfil_clientes["segmento"].isin(SEGMENTOS).all(), "Segmento fuera de los cuatro válidos"
assert perfil_clientes["segmento"].nunique() == NUM_SEGMENTOS, "Los cuatro segmentos deben tener clientes"
assert perfil_clientes.isna().sum().sum() == 0, "perfil_clientes no debe tener nulos"

assert len(afinidad_segmento_categoria) == NUM_SEGMENTOS * num_categorias, "Una fila por segmento y categoría"
assert not afinidad_segmento_categoria.duplicated(["segmento", "categoria"]).any()
assert afinidad_segmento_categoria.notna().all().all(), "afinidad_segmento_categoria no debe tener nulos"
assert (afinidad_segmento_categoria["indice_afinidad"] >= 0).all()

compras_por_segmento = connection.execute(f"""
    SELECT s.segmento, count(*) AS compras
    FROM compras c JOIN segmentos_df s ON s.cliente_id = c.cliente_id
    WHERE c.fecha_hora >= {sql_text(inicio_mes1)}::TIMESTAMP AND c.fecha_hora < {sql_text(fin_ventana)}::TIMESTAMP
    GROUP BY 1
""").df().set_index("segmento")["compras"]
peso_segmento = afinidad_segmento_categoria["segmento"].map(compras_por_segmento / compras_por_segmento.sum())
consistencia = (
    afinidad_segmento_categoria
    .assign(ponderado=afinidad_segmento_categoria["pct_compras_segmento"] * peso_segmento)
    .groupby("categoria")
    .agg(ponderado=("ponderado", "sum"), total=("pct_compras_total", "first"))
)
assert np.allclose(consistencia["ponderado"], consistencia["total"], atol=0.05), "pct_compras_total inconsistente con los segmentos"

columnas_perfil = ["cliente_id", *VARIABLES_NUMERICAS, "franja_preferida", "categoria_preferida", "segmento"]
GOLD.mkdir(parents=True, exist_ok=True)
```

```
connection.register("perfil_clientes_df", perfil_clientes[columnas_perfil])
connection.register("afinidad_df", afinidad_segmento_categoria)
connection.execute(f"COPY (SELECT * FROM perfil_clientes_df ORDER BY cliente_id) TO {sql_text(GOLD / 'perfil_clientes.parquet')} (FORMAT PARQUET, COMPRESSION ZSTD)")
connection.execute(f"COPY (SELECT * FROM afinidad_df ORDER BY list_position({segmentos_sql}, segmento), categoria) TO {sql_text(GOLD / 'afinidad_segmento_categoria.parquet')}
    (FORMAT PARQUET, COMPRESSION ZSTD)")

for name, esperado in (("perfil_clientes", num_clientes), ("afinidad_segmento_categoria", NUM_SEGMENTOS * num_categorias)):
    filas = connection.execute(f"SELECT count(*) FROM read_parquet({sql_text(GOLD / f'{name}.parquet')})").fetchone()[0]
    assert filas == esperado, f"{name}: se exportaron {filas} filas, se esperaban {esperado}"
    print(f"{name}: {filas} filas exportadas a Gold.")

sin_segmento = connection.execute(
    f"SELECT count(*) FROM read_parquet({sql_text(GOLD / 'perfil_clientes.parquet')}) WHERE segmento IS NULL"
).fetchone()[0]
assert sin_segmento == 0, "Hay clientes sin segmento en el Parquet exportado"
print("Clientes por segmento:")
print(perfil_clientes["segmento"].value_counts().reindex(SEGMENTOS).to_string())

connection.close()
```

```
perfil_clientes: 1000 filas exportadas a Gold.
afinidad_segmento_categoria: 28 filas exportadas a Gold.
Clientes por segmento:
segmento
Leales          250
Madrugadores    249
En riesgo       255
Ocasionales     246
```

Resultados

Identificamos 250 Leales, 249 Madrugadores, 255 En riesgo y 246 Ocasionales. Obtenemos una silueta de 0.4280 y un ARI mínimo de 1.0000 entre las diez semillas evaluadas. Estos indicadores muestran la separación y la estabilidad de los grupos en el escenario sintético.

Relacionamos los segmentos con las categorías que compran con mayor frecuencia relativa. Junto con las ventas por producto, esta información orienta las reglas de puntos. La agrupación describe hábitos de consumo; no estima una probabilidad de abandono. El modelo y su preprocesamiento permanecen en memoria, mientras los perfiles y afinidades se almacenan como tablas.

05 - Puntos e insights

Analizamos los perfiles de clientes, las ventas y las afinidades para definir reglas de puntos y comparar su aplicación con la política fija.

Separación temporal: utilizamos enero-febrero de 2025 para definir segmentos, rezagos y afinidades. Aplicamos los multiplicadores en marzo y calculamos las ventanas a partir del primer mes de Silver. Esta separación reserva las compras de marzo para la comparación.

Alcance: trabajamos con datos sintéticos, saldo inicial cero y las mismas compras y canjes en ambas políticas. Redondeamos por línea y acreditamos los puntos al terminar cada compra. Establecemos un tope combinado de ×3. El escenario simplifica las condiciones comerciales: no incorpora demoras de acreditación, topes diarios ni caducidad.

Código · celda 02

```
from collections import defaultdict
from decimal import Decimal, ROUND_CEILING
from hashlib import sha256
from pathlib import Path

import duckdb
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import display

PROJECT_ROOT = next((p for p in (Path.cwd(), *Path.cwd().parents)
                     if (p / "doc.md").is_file()), None)
if PROJECT_ROOT is None:
    raise RuntimeError("No se encuentra la carpeta del caso. Inicie la ejecución desde Caso-Estudio-2.")
SILVER = PROJECT_ROOT / "data" / "silver"
GOLD = PROJECT_ROOT / "data" / "gold"
PUNTOS_POR_QUETZAL = Decimal("10")
UMBRAL_AFINIDAD = 1.2
MULT_REZAGADO = Decimal("2")
TOPE_MULTIPLICADOR = Decimal("3")
SEGMENTOS = ["Leales", "Madrugadores", "En riesgo", "Ocasionales"]

def sql_text(value):
    return "'" + str(value).replace("'", "''") + "'"

entradas = [SILVER / f"{t}.parquet" for t in
            ("clientes", "productos", "restaurantes", "compras", "lineas_compra")]
entradas += [GOLD / f"{t}.parquet" for t in
             ("perfil_clientes", "ventas_productos", "afinidad_segmento_categoria")]
faltantes = [p.name for p in entradas if not p.is_file()]
if faltantes:
    raise FileNotFoundError(f"Faltan tablas de entrada: {faltantes}")
```

```
huellas_entrada = {p: sha256(p.read_bytes()).hexdigest() for p in entradas}
connection = duckdb.connect(":memory:")
connection.execute("SET threads = 1")
for path in entradas:
    connection.execute(f"CREATE VIEW {path.stem} AS SELECT * FROM read_parquet({sql_text(path)})")

perfil = connection.execute("SELECT * FROM perfil_clientes ORDER BY cliente_id").df()
if "segmento" not in perfil:
    raise ValueError("La tabla perfil_clientes requiere la columna segmento para asignar los incentivos.")
ventas = connection.execute("SELECT * FROM ventas_productos ORDER BY producto_id").df()
afinidad = connection.execute("SELECT * FROM afinidad_segmento_categoria ORDER BY segmento, categoria").df()
catalogo = connection.execute("SELECT * FROM productos ORDER BY id").df()
inicio_mes1, inicio_aplicacion, fin_aplicacion = connection.execute("""
    SELECT date_trunc('month', min(fecha_hora)),
           date_trunc('month', min(fecha_hora)) + INTERVAL 2 MONTH,
           date_trunc('month', min(fecha_hora)) + INTERVAL 3 MONTH
    FROM compras
""").fetchone()
assert inicio_mes1 is not None, "Silver no contiene compras"
assert perfil.cliente_id.is_unique and perfil.segmento.isin(SEGMENTOS).all()
assert set(perfil.cliente_id) == set(connection.execute("SELECT id FROM clientes").df().id)
assert set(ventas.producto_id) == set(catalogo.id) and ventas.producto_id.is_unique
assert not afinidad.duplicated(["segmento", "categoria"]).any()
assert set(zip(afinidad.segmento, afinidad.categoria)) == {
    (s, c) for s in SEGMENTOS for c in catalogo.categoria.unique()
}
assert afinidad.indice_afinidad.notna().all()
assert connection.execute("SELECT max(fecha_hora) FROM compras").fetchone()[0] < fin_aplicacion
print(f"Análisis: [{inicio_mes1.date()}, {inicio_aplicacion.date()})")
print(f"Aplicación: [{inicio_aplicacion.date()}, {fin_aplicacion.date()})")
print(f"DuckDB {duckdb.__version__}; clientes segmentados: {len(perfil):,}")
```

```
Análisis: [2025-01-01, 2025-03-01)
Aplicación: [2025-03-01, 2025-04-01)
DuckDB 1.5.5; clientes segmentados: 1,000
```

1. Reglas explicables

Definimos los incentivos a partir del rezago de productos, la afinidad y el comportamiento de cada segmento:

Origen	Destinatario	Multiplicador
Producto rezagado (`indice_ventas < 0.5`)	Segmentos con afinidad de su categoría > 1.2; si ninguno cumple, todos	×2
Ampliación de hábitos de Madrugadores	Hamburguesas y pollo, en cualquier franja	×2 de producto

Origen	Destinatario	Multiplicador
Reactivación de En riesgo	Todas sus líneas monetarias	×1.5 de cliente
Mayor frecuencia de Ocasionales	Todas sus líneas monetarias	×1.25 de cliente
Leales	Incentivos de productos aplicables	Sin regla adicional de cliente

Cada regla de producto identifica un producto o una categoría. Un segmento nulo indica que la regla aplica a todos los clientes. Elegimos la mayor regla de producto y como máximo una regla de cliente; en empates seleccionamos el ID menor. Limitamos el producto de ambos factores a ×3 y aplicamos las reglas únicamente durante marzo.

Código · celda 04

```
def generar_reglas(ventas_producto, afinidades):
    filas = []

    def agregar(tipo, multiplicador, motivo, producto_id=None, categoria=None, segmento=None):
        filas.append(dict(id=len(filas) + 1, tipo=tipo, producto_id=producto_id,
                          categoria=categoria, segmento=segmento,
                          multiplicador=multiplicador, motivo=motivo))

    for producto in ventas_producto.loc[ventas_producto.es_rezagado].sort_values("producto_id").itertuples():
        afines = afinidades.loc[(afinidades.categoria == producto.categoria)
                                & (afinidades.indice_afinidad > UMBRAL_AFINIDAD)]
        destinos = sorted(afines.segmento.unique()) or [None]
        for segmento in destinos:
            motivo = (f"Producto rezagado; afinidad > {UMBRAL_AFINIDAD} en {segmento}"
                      if segmento else "Producto rezagado; ningún segmento supera el umbral de afinidad")
            agregar("producto", MULT_REZAGADO, motivo,
                    producto_id=int(producto.producto_id), segmento=segmento)
    for categoria in ("hamburguesas", "pollo"):
        agregar("producto", Decimal("2"), "Ampliar categorías de consumo de Madrugadores",
                categoria=categoria, segmento="Madrugadores")
    agregar("cliente", Decimal("1.5"), "Reactivar clientes con recencia alta", segmento="En riesgo")
    agregar("cliente", Decimal("1.25"), "Incentivar frecuencia de clientes ocasionales", segmento="Ocasionales")
    reglas = pd.DataFrame(filas)
    reglas["producto_id"] = reglas.producto_id.astype("Int64")
    return reglas

reglas_puntos = generar_reglas(ventas, afinidad)
assert reglas_puntos.id.is_unique
assert all(Decimal("1") < m <= TOPE_MULTIPLICADOR for m in reglas_puntos.multiplicador)
reglas_producto = reglas_puntos.loc[reglas_puntos.tipo == "producto"]
assert (reglas_producto.producto_id.notna() ^ reglas_producto.categoria.notna()).all()
```

```
assert reglas_puntos.loc[reglas_puntos.tipo == "cliente", "segmento"].is_unique
display(reglas_puntos)
```

	id	tipo	producto_id	categoria	segmento	multiplicador	motivo
0	1	producto	4	None	Leales	2	Producto rezagado; afinidad > 1.2 en Leales
1	2	producto	8	None	Leales	2	Producto rezagado; afinidad > 1.2 en Leales
2	3	producto	24	None	Leales	2	Producto rezagado; afinidad > 1.2 en Leales
3	4	producto	28	None	En riesgo	2	Producto rezagado; afinidad > 1.2 en En riesgo
4	5	producto	<NA>	hamburguesas	Madrugadores	2	Ampliar categorías de consumo de Madrugadores
5	6	producto	<NA>	pollo	Madrugadores	2	Ampliar categorías de consumo de Madrugadores
6	7	cliente	<NA>	None	En riesgo	1.5	Reactivar clientes con recencia alta
7	8	cliente	<NA>	None	Ocasionales	1.25	Incentivar frecuencia de clientes ocasionales

2. Motor de puntos y movimientos

Calculamos los puntos base y dinámicos de cada línea monetaria de forma independiente:

puntos_base = ceil(subtotal × 10)

M = min(multiplicador_producto × multiplicador_cliente, 3)

puntos_dinámicos = ceil(subtotal × 10 × M)

Multiplicamos el importe antes del redondeo final. Por ejemplo, una línea de Q12.35 con ×1.5 genera 186 puntos. Utilizamos `Decimal` para evitar errores binarios en los importes y multiplicadores.

En enero-febrero aplicamos `M = 1` y mantenemos nulos los IDs de regla. Cada canje presenta `puntos_base = 0`, `M = 1`, reglas nulas y un débito de `precio_en_puntos × cantidad`.

Código · celda 06

```
def elegir_reglas(producto_id, categoria, segmento, reglas):
    candidatas = reglas.loc[reglas.segmento.isna() | reglas.segmento.eq(segmento)]
    coincidencia = (candidatas.producto_id.eq(producto_id).fillna(False)
                    | (candidatas.producto_id.isna() & candidatas.categoria.eq(categoria)))
    producto = candidatas.loc[candidatas.tipo.eq("producto") & coincidencia]
    cliente = candidatas.loc[candidatas.tipo.eq("cliente")]
    assert len(cliente) <= 1, "Solo puede existir una regla de cliente aplicable"

    def mejor(tabla):
        if tabla.empty:
            return None, Decimal("1")
        fila = tabla.sort_values(["multiplicador", "id"], ascending=[False, True]).iloc[0]
        return int(fila.id), Decimal(str(fila.multiplicador))

    return mejor(producto), mejor(cliente)

def calcular_puntos(subtotal, mult_producto=Decimal("1"), mult_cliente=Decimal("1"),
                    canje=False, precio_en_puntos=0, cantidad=1):
    if canje:
        return 0, Decimal("1"), -int(precio_en_puntos) * int(cantidad)
    monto = Decimal(str(subtotal))
    assert monto >= 0
    bruto = monto * PUNTOS_POR_QUETZAL
    multiplicador = min(Decimal(str(mult_producto)) * Decimal(str(mult_cliente)), TOPE_MULTIPLICADOR)
    base = int(bruto.to_integral_value(rounding=ROUND_CEILING))
    puntos = int((bruto * multiplicador).to_integral_value(rounding=ROUND_CEILING))
    return base, multiplicador, puntos

# Se conserva el subtotal DECIMAL de Silver como texto, sin pasar por float.
detalle = connection.execute("""
    SELECT l.id AS linea_id, c.id AS compra_id, c.cliente_id, c.fecha_hora,
           l.producto_id, p.nombre AS producto, p.categoria, l.cantidad,
           l.subtotal::VARCHAR AS subtotal, l.pagado_con_puntos,
           p.precio_en_puntos, pc.segmento
    FROM lineas_compra l
    JOIN compras c ON c.id = l.compra_id
    JOIN productos p ON p.id = l.producto_id
    JOIN perfil_clientes pc ON pc.cliente_id = c.cliente_id
    ORDER BY c.fecha_hora, c.id, l.pagado_con_puntos DESC, l.id
""").df()
```

```
assert len(detalle) == connection.execute("SELECT count(*) FROM lineas_compra").fetchone()[0]
assert detalle.linea_id.is_unique
assert detalle.fecha_hora.between(inicio_aplicacion, fin_aplicacion, inclusive="left").any()

seleccion = {(int(p.id), s): elegir_reglas(int(p.id), p.categoria, s, reglas_puntos)
             for p in catalogo.itertuples() for s in SEGMENTOS}
filas_movimientos = []
for linea in detalle.itertuples():
    regla_producto, mult_producto = None, Decimal("1")
    regla_cliente, mult_cliente = None, Decimal("1")
    if inicio_aplicacion <= linea.fecha_hora < fin_aplicacion and not linea.pagado_con_puntos:
        (regla_producto, mult_producto), (regla_cliente, mult_cliente) = seleccion[(linea.producto_id, linea.segmento)]
    base, multiplicador, puntos = calcular_puntos(
        linea.subtotal, mult_producto, mult_cliente, linea.pagado_con_puntos,
        linea.precio_en_puntos, linea.cantidad,
    )
    filas_movimientos.append(dict(
        id=len(filas_movimientos) + 1, cliente_id=linea.cliente_id, linea_id=linea.linea_id,
        regla_producto_id=regla_producto, regla_cliente_id=regla_cliente,
        tipo="canje" if linea.pagado_con_puntos else "acumulacion", fecha_hora=linea.fecha_hora,
        puntos_base=base, multiplicador_aplicado=multiplicador, puntos=puntos,
    ))
movimientos_puntos = pd.DataFrame(filas_movimientos)
for columna in ("regla_producto_id", "regla_cliente_id"):
    movimientos_puntos[columna] = movimientos_puntos[columna].astype("Int64")
display(movimientos_puntos.head(8))
print(f"Movimientos: {len(movimientos_puntos):,}; uno por línea de Silver.")
```

	id	cliente_id	linea_id	regla_producto_id	regla_cliente_id	tipo	fecha_hora	puntos_base	multiplicador_aplicado	puntos
0	1	584	1	<NA>	<NA>	acumulacion	2025-01-01 06:00:40	245	1	245
1	2	584	2	<NA>	<NA>	acumulacion	2025-01-01 06:00:40	310	1	310
2	3	584	3	<NA>	<NA>	acumulacion	2025-01-01 06:00:40	110	1	110
3	4	394	4	<NA>	<NA>	acumulacion	2025-01-01 06:03:32	260	1	260
4	5	627	5	<NA>	<NA>	acumulacion	2025-01-01 06:04:26	180	1	180

	id	cliente_id	linea_id	regla_producto_id	regla_cliente_id	tipo	fecha_hora	puntos_base	multiplicador_aplicado	puntos
5	6	627	6	<NA>	<NA>	acumulacion	2025-01-01 06:04:26	778	1	778
6	7	627	7	<NA>	<NA>	acumulacion	2025-01-01 06:04:26	360	1	360
7	8	422	8	<NA>	<NA>	acumulacion	2025-01-01 06:09:12	155	1	155

Movimientos: 49,525; uno por línea de Silver.

3. Validaciones funcionales y saldos

Verificamos el cálculo de puntos, la selección de reglas superpuestas, los desempates y la aplicación general cuando no existe afinidad alta. Revisamos las claves, los signos, la vigencia y el tope en todo el historial.

Para cada compra descontamos todos los canjes antes de acreditar puntos, tanto en la política fija como en la dinámica. Así comprobamos que el cliente dispone de saldo suficiente sin utilizar puntos de la misma compra. Seguimos el orden de fecha e ID de compra; el ID resuelve las compras que comparten una marca de tiempo.

Código · celda 08

```
casos = [
    ("Q50 sin multiplicadores", dict(subtotal="50"), (500, Decimal("1"), 500)),
    ("Q12.35 y redondeo", dict(subtotal="12.35"), (124, Decimal("1"), 124)),
    ("Producto Q15 ×2", dict(subtotal="15", mult_producto="2"), (150, Decimal("2"), 300)),
    ("Producto ×2 y cliente ×1.5", dict(subtotal="15", mult_producto="2", mult_cliente="1.5"), (150, Decimal("3"), 450)),
    ("Tope: ×2 por ×2", dict(subtotal="15", mult_producto="2", mult_cliente="2"), (150, Decimal("3"), 450)),
    ("Redondeo final sin doble redondeo", dict(subtotal="12.35", mult_cliente="1.5"), (124, Decimal("1.5"), 186)),
    ("Canje de dos unidades", dict(subtotal="0", canje=True, precio_en_puntos=500, cantidad=2), (0, Decimal("1"), -1000)),
]
for nombre, argumentos, esperado in casos:
    assert calcular_puntos(**argumentos) == esperado, nombre

reglas_prueba = pd.DataFrame([
    dict(id=1, tipo="producto", producto_id=None, categoria="pollo", segmento=None, multiplicador=Decimal("1.5")),
    dict(id=2, tipo="producto", producto_id=99, categoria=None, segmento="Leales", multiplicador=Decimal("2")),
    dict(id=3, tipo="producto", producto_id=None, categoria="pollo", segmento="Leales", multiplicador=Decimal("2")),
    dict(id=4, tipo="cliente", producto_id=None, categoria=None, segmento="Leales", multiplicador=Decimal("1.25")),
])
reglas_prueba["producto_id"] = reglas_prueba.producto_id.astype("Int64")
assert elegir_reglas(99, "pollo", "Leales", reglas_prueba) == ((2, Decimal("2")), (4, Decimal("1.25")))
```

```
assert elegir_reglas(99, "pollo", "Ocasionales", reglas_prueba) == ((1, Decimal("1.5")), (None, Decimal("1")))
sin_afinidad = afinidad.assign(indice_afinidad=1.2)
generales = generar_reglas(ventas, sin_afinidad).query("producto_id.notna()")
assert len(generales) == int(ventas.es_rezagado.sum()) and generales.segmento.isna().all()

auditoria = detalle.merge(movimientos_puntos, on=["linea_id", "cliente_id", "fecha_hora"], validate="one_to_one")
es_canje = auditoria.tipo.eq("canje")
es_analisis = auditoria.fecha_hora.lt(inicio_aplicacion)
assert movimientos_puntos.id.is_unique and movimientos_puntos.linea_id.is_unique
assert set(movimientos_puntos.linea_id) == set(detalle.linea_id)
assert (auditoria.loc[es_canje, "puntos"] == -auditoria.loc[es_canje, "precio_en_puntos"] * auditoria.loc[es_canje, "cantidad"]).all()
assert auditoria.loc[es_canje, "puntos_base"].eq(0).all()
assert auditoria.loc[~es_canje, "puntos"].ge(auditoria.loc[~es_canje, "puntos_base"]).all()
assert auditoria.loc[~es_canje & es_analisis, "puntos"].eq(auditoria.loc[~es_canje & es_analisis, "puntos_base"]).all()
assert auditoria.loc[es_canje | es_analisis, ["regla_producto_id", "regla_cliente_id"]].isna().all().all()
assert auditoria.loc[es_canje | es_analisis, "multiplicador_aplicado"].eq(Decimal("1")).all()
assert auditoria.multiplicador_aplicado.between(Decimal("1"), TOPE_MULTIPLICADOR).all()
for columna, tipo in (("regla_producto_id", "producto"), ("regla_cliente_id", "cliente")):
    assert set(auditoria[columna].dropna()) <= set(reglas_puntos.loc[reglas_puntos.tipo.eq(tipo), "id"])

auditoria["puntos_fijos"] = np.where(es_canje, auditoria.puntos, auditoria.puntos_base)
auditoria["puntos_extra"] = np.where(es_canje, 0, auditoria.puntos - auditoria.puntos_base)
por_compra = auditoria.groupby(["fecha_hora", "compra_id", "cliente_id"], sort=True).agg(
    canje=("puntos", lambda puntos: int(puntos[puntos < 0].sum())),
    acumulacion_fija=("puntos_base", "sum"),
    acumulacion_dinamica=("puntos", lambda puntos: int(puntos[puntos >= 0].sum())),
).reset_index()
saldos_fijos, saldos_dinamicos = defaultdict(int), defaultdict(int)
for compra in por_compra.itertuples():
    cliente = compra.cliente_id
    assert saldos_fijos[cliente] + compra.canje >= 0, f"Saldo fijo insuficiente: compra {compra.compra_id}"
    assert saldos_dinamicos[cliente] + compra.canje >= 0, f"Saldo dinámico insuficiente: compra {compra.compra_id}"
    saldos_fijos[cliente] += int(compra.canje + compra.acumulacion_fija)
    saldos_dinamicos[cliente] += int(compra.canje + compra.acumulacion_dinamica)
saldos = perfil[["cliente_id", "segmento"]].copy()
saldos["saldo_fijo"] = saldos.cliente_id.map(lambda c: saldos_fijos[c])
saldos["saldo_dinamico"] = saldos.cliente_id.map(lambda c: saldos_dinamicos[c])
assert saldos.saldo_dinamico.ge(saldos.saldo_fijo).all()
assert saldos.saldo_dinamico.sum() == movimientos_puntos.puntos.sum()
assert saldos.saldo_fijo.sum() == auditoria.puntos_fijos.sum()
print(f"{len(casos)} ejemplos de cálculo, selección de reglas y fallback: correctos.")
print(f"Historial de {len(por_compra):,} compras: canjes financiados con saldo previo en ambas políticas.")
print(f"Saldo final mínimo: fijo {saldos.saldo_fijo.min():,}; dinámico {saldos.saldo_dinamico.min():,} puntos.")
```

```
7 ejemplos de cálculo, selección de reglas y fallback: correctos.
Historial de 20,000 compras: canjes financiados con saldo previo en ambas políticas.
Saldo final mínimo: fijo 150; dinámico 150 puntos.
```

4. Exportación a Gold

Guardamos las reglas y los movimientos en Parquet, con identificadores de regla anulables, puntos enteros y multiplicadores `DECIMAL(6,2)`. Comprobamos el esquema y el contenido de los archivos para asegurar que representan correctamente los resultados. Cada ejecución actualiza las tablas sin acumular registros duplicados.

Código · celda 10

```
connection.register("reglas_df", reglas_puntos)
connection.register("movimientos_df", movimientos_puntos)
connection.execute("""
    CREATE TABLE salida_reglas AS
    SELECT id::INTEGER AS id, tipo::VARCHAR AS tipo, producto_id::INTEGER AS producto_id,
           categoria::VARCHAR AS categoria, segmento::VARCHAR AS segmento,
           multiplicador::DECIMAL(6,2) AS multiplicador, motivo::VARCHAR AS motivo
    FROM reglas_df ORDER BY id
""")
connection.execute("""
    CREATE TABLE salida_movimientos AS
    SELECT id::INTEGER AS id, cliente_id::INTEGER AS cliente_id, linea_id::INTEGER AS linea_id,
           regla_producto_id::INTEGER AS regla_producto_id, regla_cliente_id::INTEGER AS regla_cliente_id,
           tipo::VARCHAR AS tipo, fecha_hora::TIMESTAMP AS fecha_hora,
           puntos_base::BIGINT AS puntos_base,
           multiplicador_aplicado::DECIMAL(6,2) AS multiplicador_aplicado, puntos::BIGINT AS puntos
    FROM movimientos_df ORDER BY id
""")
GOLD.mkdir(parents=True, exist_ok=True)
for tabla, origen in (("reglas_puntos", "salida_reglas"), ("movimientos_puntos", "salida_movimientos")):
    ruta = sql_text(GOLD / f"{tabla}.parquet")
    connection.execute(f"COPY {origen} TO {ruta} (FORMAT PARQUET, COMPRESSION ZSTD)")
    connection.execute(f"CREATE VIEW releida_{tabla} AS SELECT * FROM read_parquet({ruta})")
    assert connection.execute(f"DESCRIBE {origen}").fetchall() == connection.execute(f"DESCRIBE releida_{tabla}").fetchall()
    for izquierda, derecha in ((origen, f"releida_{tabla}"), (f"releida_{tabla}", origen)):
        assert connection.execute(f"SELECT count(*) FROM (SELECT * FROM {izquierda} EXCEPT ALL SELECT * FROM {derecha})").fetchone()[0] == 0
    cantidad = connection.execute(f"SELECT count(*) FROM {origen}").fetchone()[0]
    print(f"{tabla}.parquet: {cantidad:,} filas; esquema y contenido verificados.")
assert huellas_entrada == {p: sha256(p.read_bytes()).hexdigest() for p in entradas}
connection.close()
print("Exportación y validación completas.")
```

```
reglas_puntos.parquet: 8 filas; esquema y contenido verificados.

movimientos_puntos.parquet: 49,525 filas; esquema y contenido verificados.
Exportación y validación completas.
```

5. Clientes, productos rezagados y afinidad

Describimos los segmentos mediante su tamaño y sus promedios de enero-febrero. Comparamos las ventas de cada producto con el promedio de su categoría para identificar los rezagados.

Medimos la afinidad mediante la presencia de cada categoría en las compras, incluidos los canjes. Una compra puede contener varias categorías, por lo que los porcentajes no representan categorías excluyentes ni suman necesariamente 100 %.

Código · celda 12

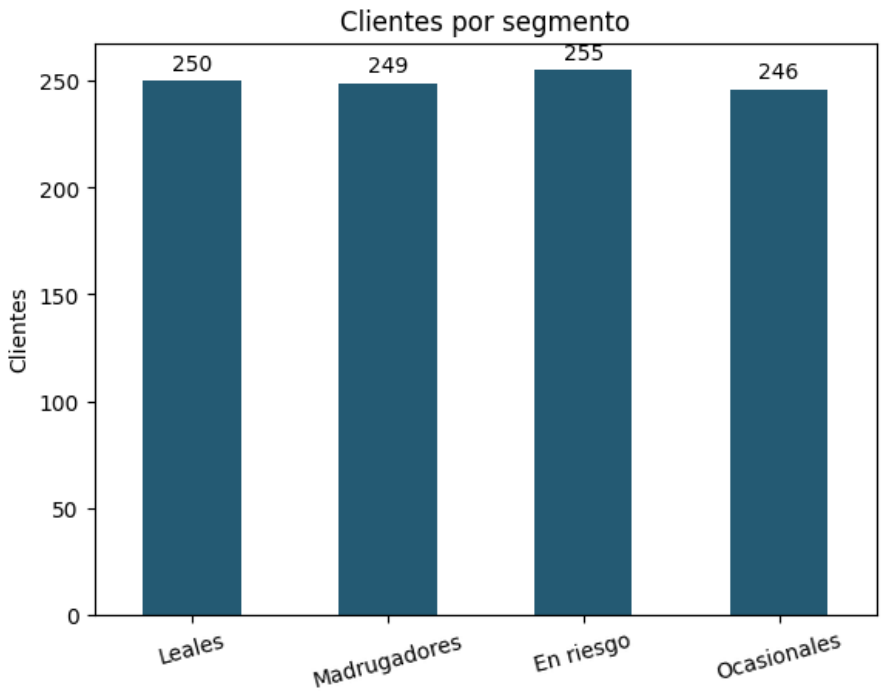
```
resumen_segmentos = perfil.groupby("segmento").agg(
    clientes=("cliente_id", "size"), recencia_dias=("recencia_dias", "mean"),
    compras_por_mes=("frecuencia_mensual", "mean"), ticket_promedio_Q=("gasto_promedio", "mean"),
).reindex(SEGMENTOS)
rezagados = ventas.loc[ventas.es_rezagado].merge(
    catalogo[["id", "nombre"]].rename(columns={"id": "producto_id"}), on="producto_id", validate="one_to_one"
)
matriz_afinidad = afinidad.pivot(index="segmento", columns="categoria", values="indice_afinidad").reindex(SEGMENTOS)
display(resumen_segmentos.round(2))
display(rezagados[["nombre", "categoria", "unidades_vendidas", "indice_ventas"]])

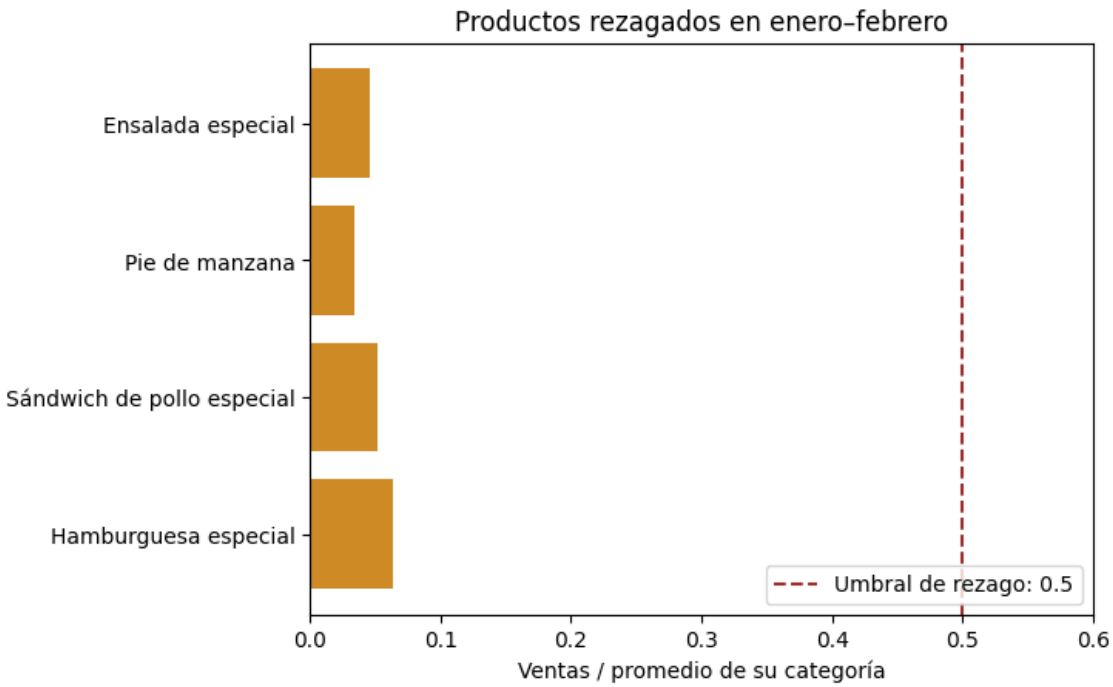
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5), layout="constrained")
resumen_segmentos.clientes.plot.bar(ax=axes[0], color="#245A73", rot=15)
axes[0].set(title="Clientes por segmento", xlabel="", ylabel="Clientes")
axes[0].bar_label(axes[0].containers[0], padding=3)
if not rezagados.empty:
    axes[1].barh(rezagados.nombre, rezagados.indice_ventas, color="#CE8B25")
axes[1].axvline(0.5, color="#9E3030", linestyle="--", label="Umbral de rezago: 0.5")
axes[1].set(title="Productos rezagados en enero-febrero", xlabel="Ventas / promedio de su categoría", xlim=(0, 0.6))
axes[1].legend(loc="lower right")
plt.show()

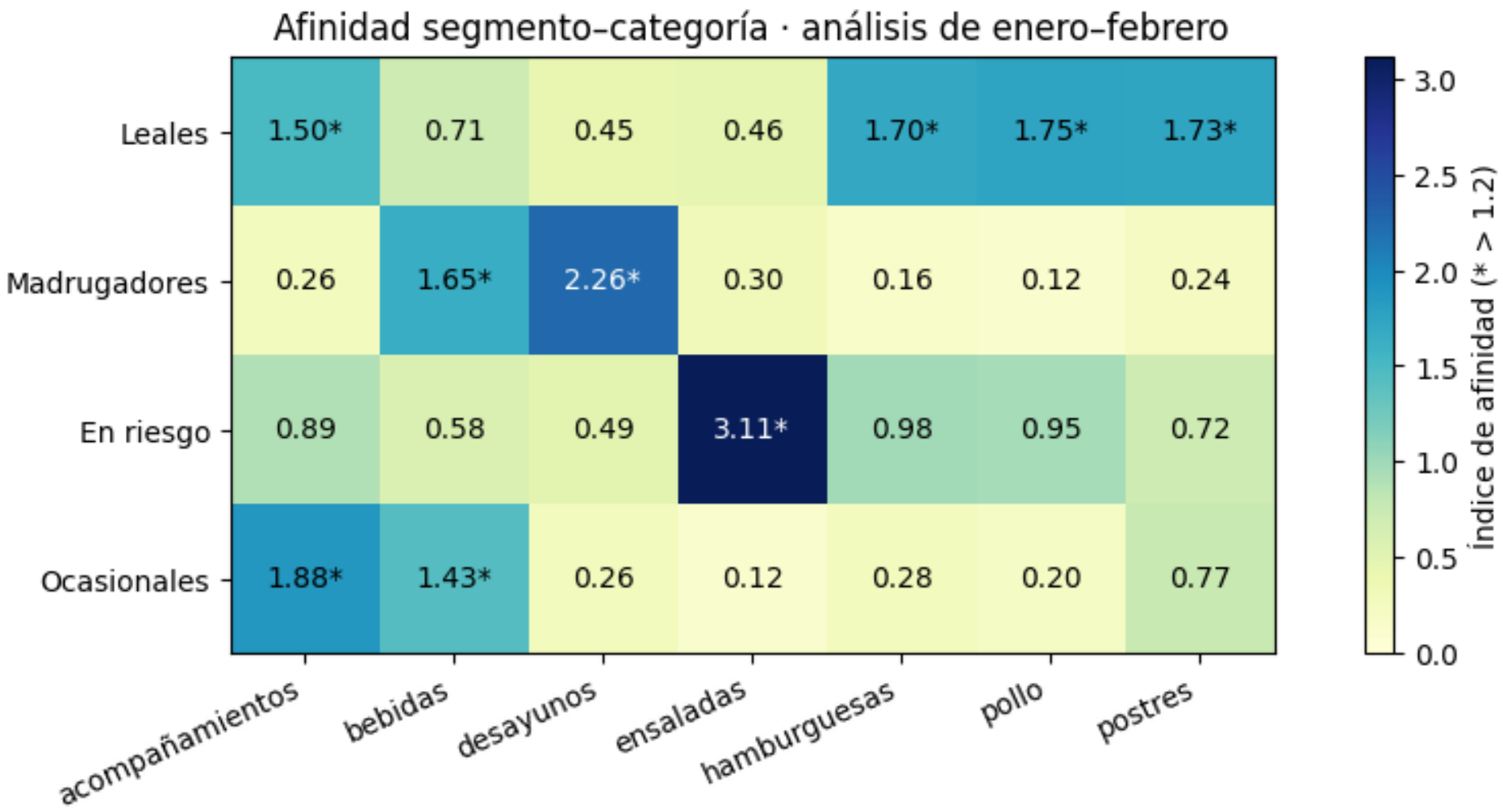
fig, ax = plt.subplots(figsize=(10, 4), layout="constrained")
mapa = ax.imshow(matriz_afinidad.to_numpy(), cmap="YlGnBu", vmin=0)
ax.set_xticks(range(len(matriz_afinidad.columns)), matriz_afinidad.columns, rotation=25, ha="right")
ax.set_yticks(range(len(SEGMENTOS)), SEGMENTOS)
ax.set_title("Afinidad segmento-categoría · análisis de enero-febrero")
for i in range(len(SEGMENTOS)):
    for j in range(len(matriz_afinidad.columns)):
        valor = matriz_afinidad.iloc[i, j]
        ax.text(j, i, f"{valor:.2f}" + ("*" if valor > UMBRAL_AFINIDAD else ""),
                ha="center", va="center", color="white" if valor > 2 else "black")
fig.colorbar(mapa, ax=ax, label="Índice de afinidad (* > 1.2)")
plt.show()
```

	clientes	recencia_dias	compras_por_mes	ticket_promedio_Q
segmento				
Leales	250	1.94	12.00	102.68
Madrugadores	249	2.79	8.69	39.12
En riesgo	255	26.08	6.38	58.78
Ocasionales	246	11.25	1.36	26.41

	nombre	categoria	unidades_vendidas	indice_ventas
0	Hamburguesa especial	hamburguesas	142	0.0635
1	Sándwich de pollo especial	pollo	95	0.0521
2	Pie de manzana	postres	44	0.0343
3	Ensalada especial	ensaladas	39	0.0462


Clientes por segmento
Clientes
250
249
255
246
0
50
100
150
200
250
Leales
Madrugadores
En riesgo
Ocasionales


Productos rezagados en enero–febrero
Ensalada especial
Pie de manzana
Sándwich de pollo especial
Hamburguesa especial
Umbral de rezago: 0.5
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Ventas / promedio de su categoría



6. Comparación de políticas en marzo

Comparamos la emisión de puntos a partir de las acumulaciones: `puntos_extra = puntos_dinámicos - puntos_base`. Reportamos los canjes por separado, ya que representan el mismo débito en ambas políticas.

Calculamos el porcentaje de incremento con los puntos base de marzo como denominador. Este indicador expresa emisión adicional de puntos, no costo monetario.

Código · celda 14

```
marzo = auditoria.loc[auditoria.fecha_hora.between(inicio_aplicacion, fin_aplicacion, inclusive="left")].copy()
acumulaciones = marzo.loc[marzo.tipo.eq("acumulacion")].copy()
base_marzo = int(acumulaciones.puntos_base.sum())
dinamicos_marzo = int(acumulaciones.puntos.sum())
extras_marzo = int(acumulaciones.puntos_extra.sum())
canjes_marzo = -int(marzo.loc[marzo.tipo.eq("canje"), "puntos"].sum())
incremento_pct = 100 * extras_marzo / base_marzo if base_marzo else 0.0
beneficiados = int(acumulaciones.loc[acumulaciones.puntos_extra.gt(0), "cliente_id"].nunique())
clientes_marzo = int(marzo.cliente_id.nunique())
assert dinamicos_marzo - base_marzo == extras_marzo
comparacion = pd.DataFrame({
    "Política": ["Fija", "Dinámica"],
    "Puntos emitidos": [base_marzo, dinamicos_marzo],
    "Puntos canjeados": [canjes_marzo, canjes_marzo],
    "Movimiento neto del mes": [base_marzo - canjes_marzo, dinamicos_marzo - canjes_marzo],
})
display(comparacion)
print(f"Emisión adicional: {extras_marzo:,} puntos (+{incremento_pct:.2f}%).")
print(f"Clientes con puntos extra: {beneficiados:,} de {clientes_marzo:,} activos en marzo.")

costo_segmento = acumulaciones.groupby("segmento").agg(
    puntos_base=("puntos_base", "sum"), puntos_dinamicos=("puntos", "sum"),
    puntos_extra=("puntos_extra", "sum"),
).reindex(SEGMENTOS, fill_value=0)
costo_segmento["incremento_pct"] = 100 * costo_segmento.puntos_extra / costo_segmento.puntos_base.replace(0, np.nan)
costo_segmento["participacion_extra_pct"] = 100 * costo_segmento.puntos_extra / extras_marzo if extras_marzo else 0.0
costo_producto = acumulaciones.groupby(["producto_id", "producto", "categoria"]).agg(
    puntos_base=("puntos_base", "sum"), puntos_dinamicos=("puntos", "sum"), puntos_extra=("puntos_extra", "sum"),
).reset_index().sort_values(["puntos_extra", "producto_id"], ascending=[False, True])
assert int(costo_segmento.puntos_extra.sum()) == extras_marzo == int(costo_producto.puntos_extra.sum())
display(costo_segmento.round(2))
display(costo_producto.head(10))

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5), layout="constrained")
costo_segmento[["puntos_base", "puntos_dinamicos"]].div(1000).plot.bar(
    ax=axes[0], color=["#8B9DA5", "#245A73"], rot=15
)
axes[0].set(title="Emisión de marzo por segmento", xlabel="", ylabel="Miles de puntos")
axes[0].legend(["Regla fija", "Reglas dinámicas"])
top_productos = costo_producto.head(8).sort_values("puntos_extra")
axes[1].barh(top_productos.producto, top_productos.puntos_extra, color="#CE8B25")
axes[1].set(title="Productos con mayor emisión adicional", xlabel="Puntos extra en marzo")
plt.show()
```

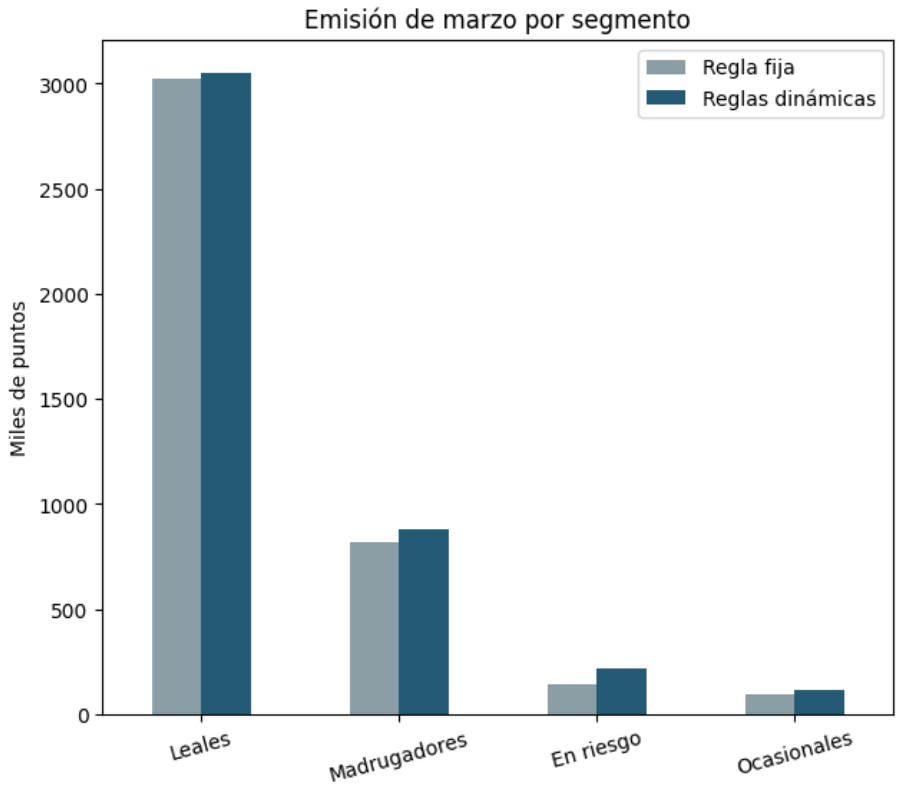
	Política	Puntos emitidos	Puntos canjeados	Movimiento neto del mes
0	Fija	4080306	1417116	2663190
1	Dinámica	4271226	1417116	2854110

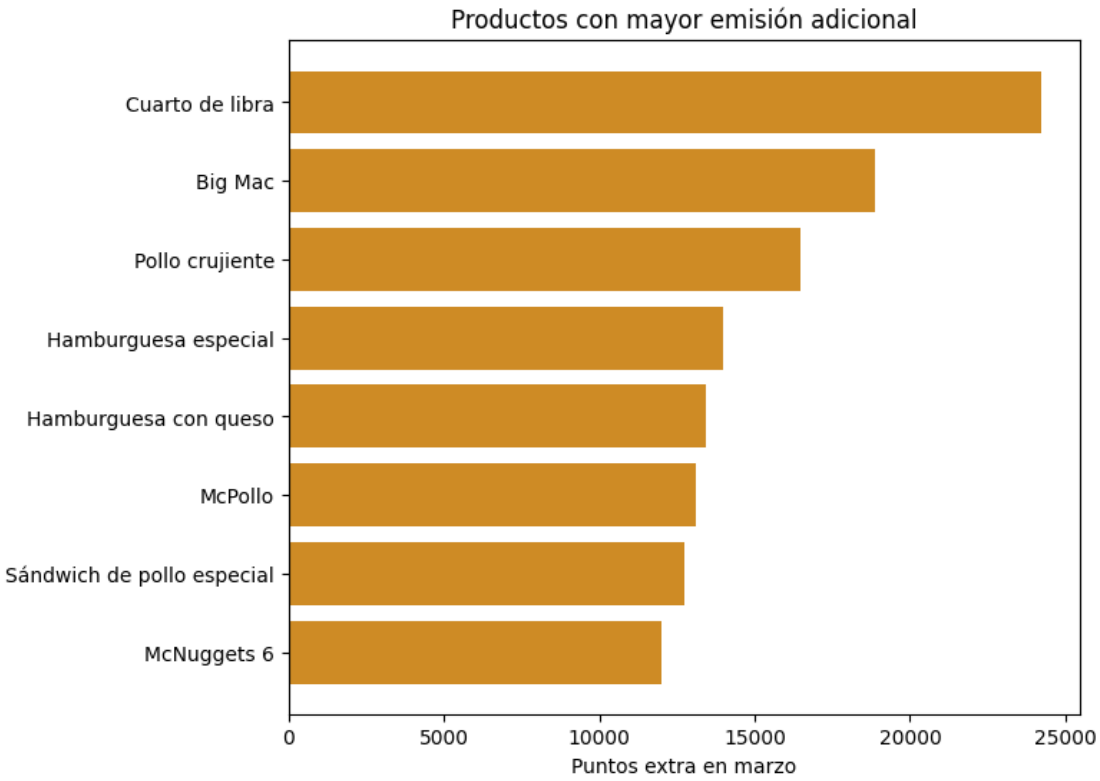
```
Emisión adicional: 190,920 puntos (+4.68%).
Clientes con puntos extra: 605 de 907 activos en marzo.
```

	puntos_base	puntos_dinamicos	puntos_extra	incremento_pct	participacion_extra_pct
segmento					
Leales	3024031	3053660	29629	0.98	15.52
Madrugadores	819063	882746	63683	7.78	33.36
En riesgo	142790	216737	73947	51.79	38.73
Ocasionales	94422	118083	23661	25.06	12.39

	producto_id	producto	categoria	puntos_base	puntos_dinamicos	puntos_extra
1	2	Cuarto de libra	hamburguesas	495586	519829	24243
0	1	Big Mac	hamburguesas	427767	446635	18868
6	7	Pollo crujiente	pollo	372750	389206	16456
3	4	Hamburguesa especial	hamburguesas	13975	27950	13975
2	3	Hamburguesa con queso	hamburguesas	236880	250290	13410
4	5	McPollo	pollo	322560	335680	13120
7	8	Sándwich de pollo especial	pollo	12942	25681	12739
5	6	McNuggets 6	pollo	294975	306967	11992
24	25	Ensalada de pollo	ensaladas	79100	89165	10065

	producto_id	producto	categoria	puntos_base	puntos_dinamicos	puntos_extra
26	27	Ensalada mixta	ensaladas	60180	68172	7992





7. Interpretación de resultados y evaluación

Interpretamos las diferencias entre políticas sobre un conjunto fijo de compras. Los clientes sintéticos mantienen su comportamiento, por lo que los puntos adicionales permiten dimensionar la emisión, pero no medir un aumento de ventas.

Planteamos evaluar el efecto comercial mediante un piloto con asignación aleatoria a tratamiento y control, presupuesto de puntos y seguimiento de frecuencia, ticket, ventas de rezagados, canjes y margen. La conversión de puntos a costo en quetzales requiere conocer el margen y el costo de los productos canjeados.

Resultados de la ejecución

- Generamos **8 reglas** y **49,525 movimientos**, con trazabilidad a la línea y a las reglas aplicadas.
- Identificamos **4 productos rezagados**. La afinidad dirige sus incentivos a los segmentos que ya eligen su categoría; las reglas de Madrugadores exploran categorías distintas a su hábito principal.
- En marzo, la regla fija emite **4,080,306 puntos** y la dinámica **4,271,226**: **190,920 puntos adicionales (+4.68%)**.
- **605 de 907 clientes activos** reciben puntos extra. **En riesgo** concentra el **38.73%** de la emisión adicional; conviene revisar esta concentración al definir el presupuesto del piloto.
- Los canjes de marzo consumen **1,417,116 puntos** en ambas políticas. Todos se financian con saldo previo; el saldo final mínimo es **150 puntos** bajo la política dinámica.
- La POC verifica el cálculo y la trazabilidad. **No demuestra un aumento de ventas ni rentabilidad** y no representa todas las restricciones del programa real.

El flujo integra la generación de datos, la limpieza, la segmentación y el cálculo de puntos.

Repositorio y reproducción

Repositorio de GitHub.

El proyecto incluye los cinco notebooks ejecutables, los datos de la simulación, las dependencias y las pruebas de integridad. La ejecución sigue el orden generación, limpieza, preparación de Gold, segmentación y cálculo de puntos.